



디지털공학

9 주차



컨트롤로직 설계

10 주차

8 레지스터 전송 레벨 설계

❖ 개요

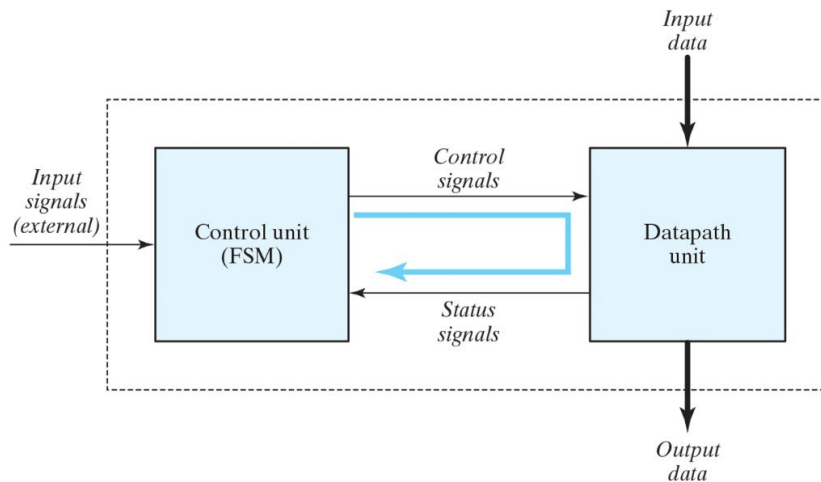
- 디지털 시스템은 플립플롭과 게이트로 만든 순차 논리 시스템을 칭함
- 순차회로의 동작은 앞서 강의한 상태표 및 상태 천이도를 이용하여 표현 가능
- 대형 디지털 시스템의 경우 상태의 수가 너무 많아 표현에 어려움이 있어 기능에 따른 모듈을 구성하고 모듈 간 Data Path, Control Signal을 이용하여 시스템 표현
- 상기 모듈은 레지스터, 디코더, 멀티플렉서, 산술소자 및 제어 논리와 같은 기본 디지털 소자를 이용하여 구성
- 레지스터 전송 레벨 설계란 위의 모듈을 구성하고 있는 레지스터에 저장된 2진 정보 처리 및 흐름 단계에서의 설계를 의미

❖ 상태 머신(SM, State Machine)

- 시스템을 추상화 해서 표현하는 수학적 모델링 방법
- 대다수의 시스템의 경우 상태의 수가 유한한 개수를 가지는 유한 상태 머신(FSM, Finite State Machine)의 형태를 가져 SM은 일반적으로 FSM을 칭함
- 외부의 입력에 따라 한 상태에서 다른 상태로 천이(Transition)하게 되며, 천이를 발생시키는 입력의 처리 방법에 따라 Event, Condition 으로 구분
- 컴퓨터 시스템이 적용되는 장치에서 활용되며, 엘리베이터, 교통신호등, 자판기, 비밀번호 패턴 등 다양한 분야에서 활용되고 있으며, 소프트웨어 설계에도 역시 활용되고 있음

❖ 유한 상태 머신(FSM, Finite State Machine)

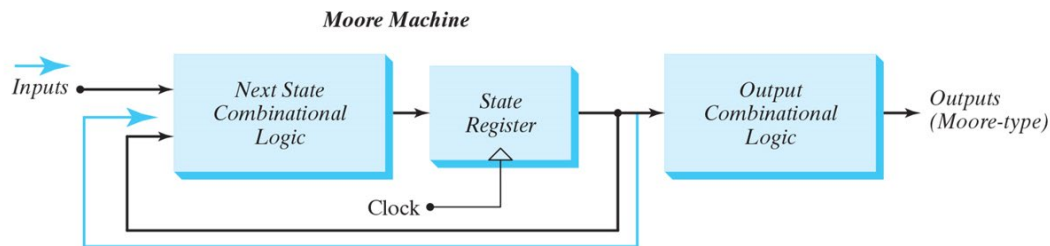
- 디지털 시스템은 크게 데이터 유닛과 제어 유닛으로 구분
- 디지털 시스템의 논리설계는 데이터 처리 연산을 위한 회로설계, 다양한 연산의 수행 순서를 정하는 제어회로 설계가 주요 설계 대상
 - Datapath Unit : 시스템 요구사항에 따라 레지스터 데이터 조작
 - Control Unit : 데이터의 처리 및 흐름제어를 위한 명령
- FSM은 Datapath Unit의 연산 순서를 제어하는 신호를 생성하는 순차회로를 의미함



❖ FSM의 표현 방법

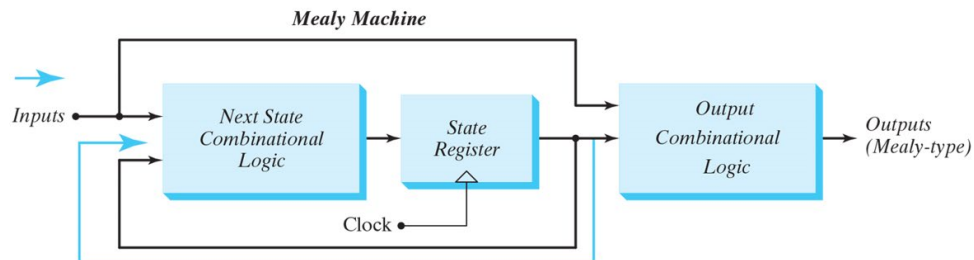
• Moore Machine

- ✓ 출력 값이 현재 상태 값에 의해서만 결정되는 모델
- ✓ 상태를 나타내는 플립플롭의 출력에만 의존하므로 클럭과 동기화 됨

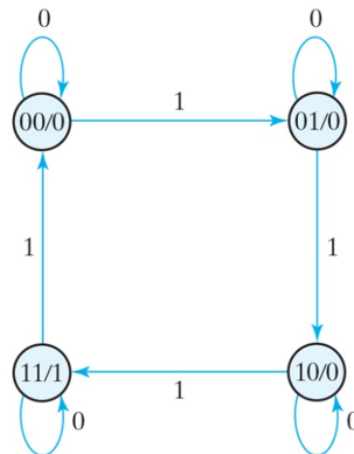
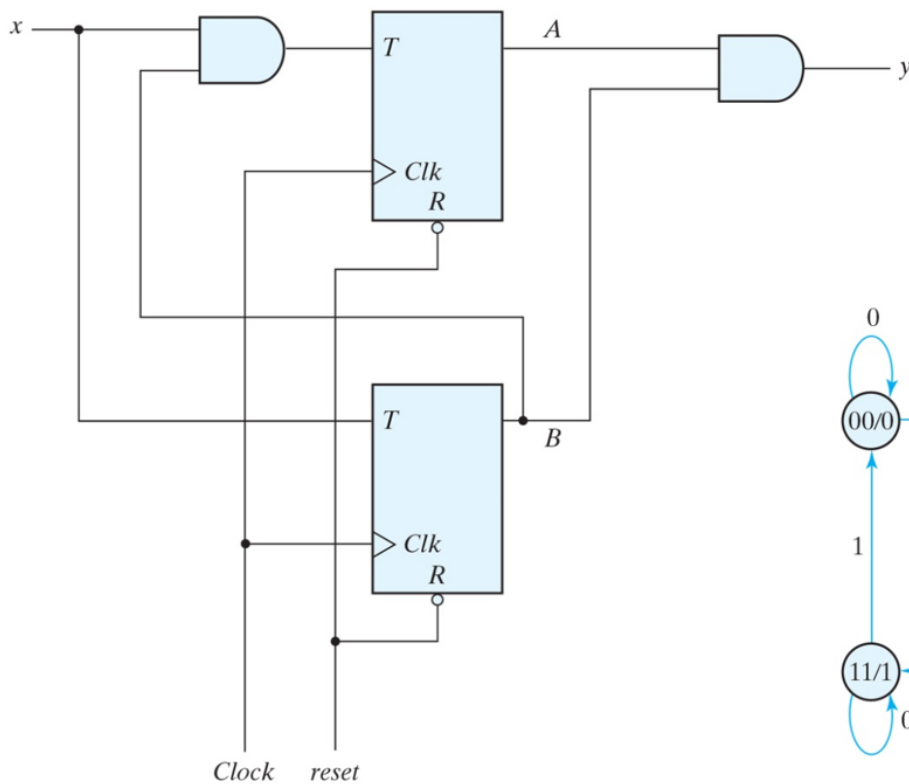


• Mealy Machine

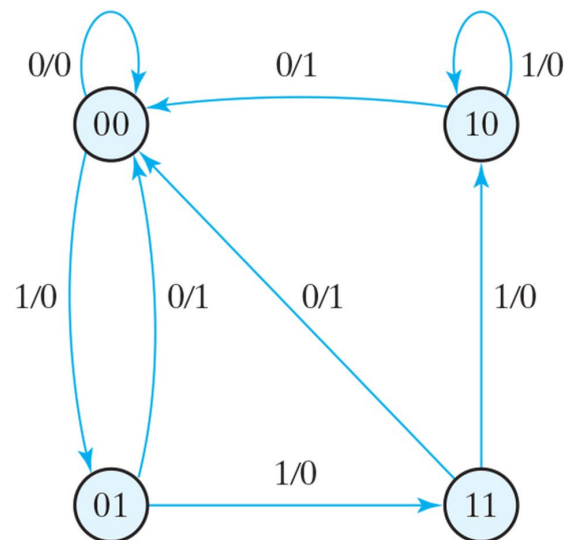
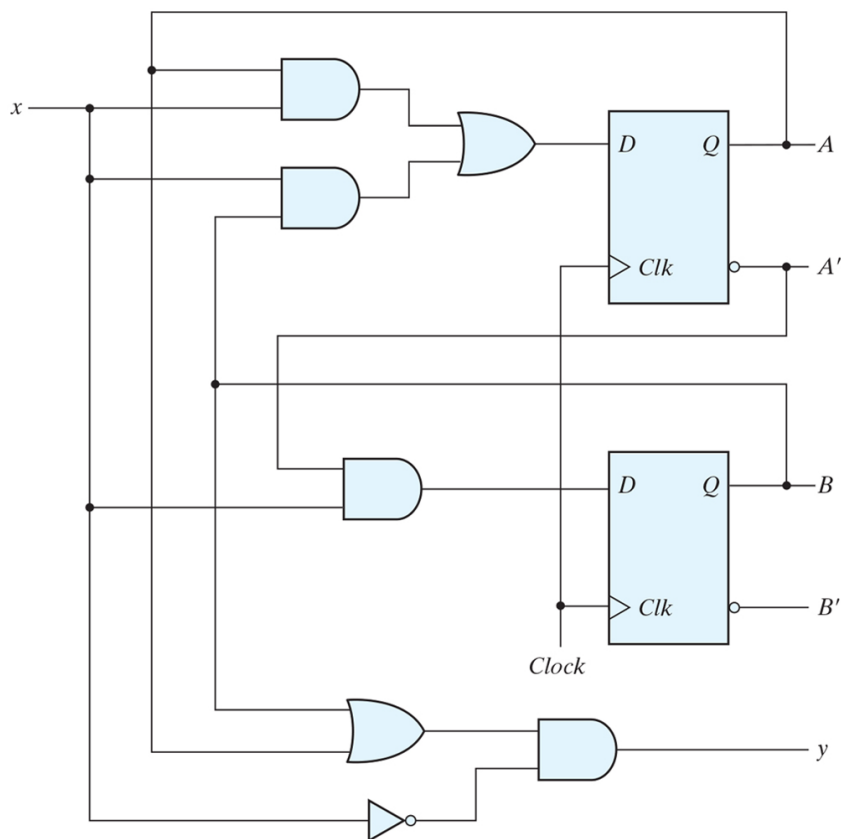
- ✓ 출력 값이 현재상태 + 입력신호의 조합에 의해 결정되는 모델
- ✓ 비동기 입력신호가 사용될 경우 입력신호의 천이와 클럭 사이클 간의 시간 지연으로 글리치(glitch, 하드웨어에 의한 시스템의 일시적 오류)가 발생할 수 있음
- ✓ 동기화 시스템 설계 시 글리치 문제 해결을 위해 시스템 클럭의 활성화 에지 직전의 비동기 입력값을 샘플링하여 사용



- Moore Machine

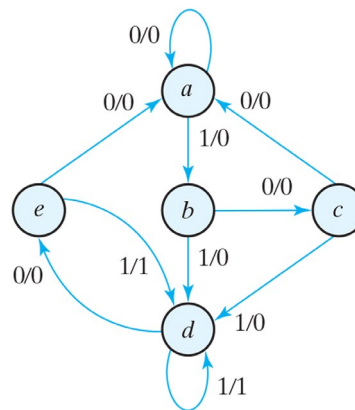


- Mealy Machine



❖ FSM 설계

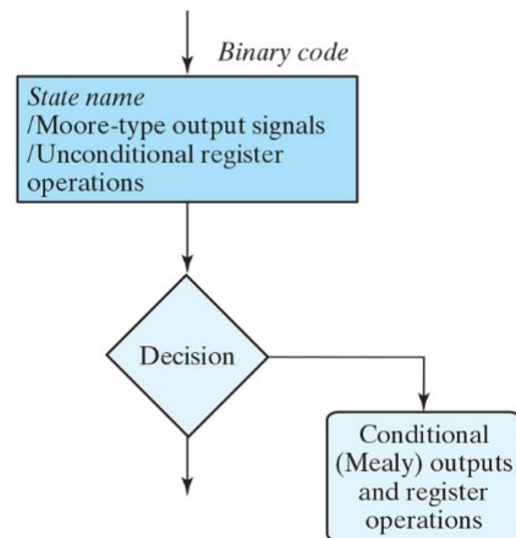
1. 시스템 사양 및 기능 분석을 통해 상태도 도출
2. 상태축소 가능 시 축소
3. 상태에 대한 2진 코드 할당
4. 2진 코드 상태표 작성
5. 플립플롭 종류 선택
6. 선택한 플립플롭에 따른 논리식 도출
7. 논리회로 작성(상태머신)

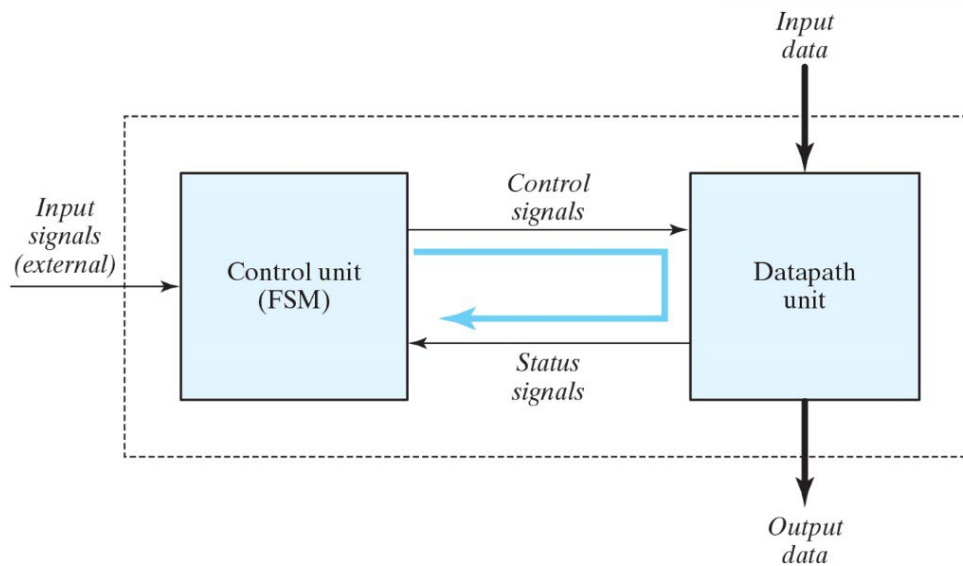
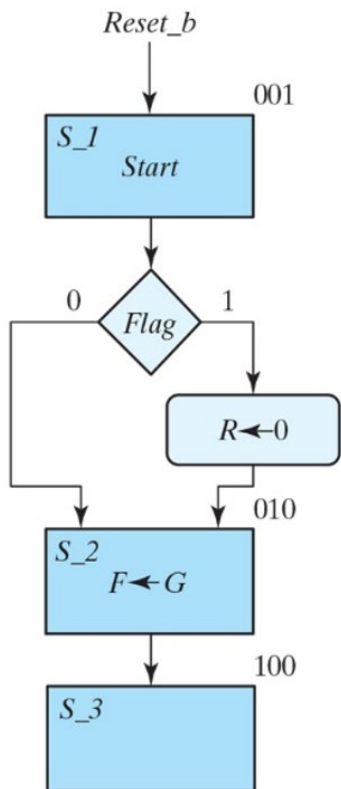


Present State		Input x	Next State		Flip-Flop Inputs			
A	B		A	B	J_A	K_A	J_B	K_B
0	0	0	0	1	0	0	1	0
0	0	1	0	0	0	0	0	1
0	1	0	1	1	1	1	1	0
0	1	1	1	0	1	0	0	1
1	0	0	1	1	0	0	1	1
1	0	1	1	0	0	0	0	0
1	1	0	0	0	1	1	1	1
1	1	1	1	1	1	0	0	0

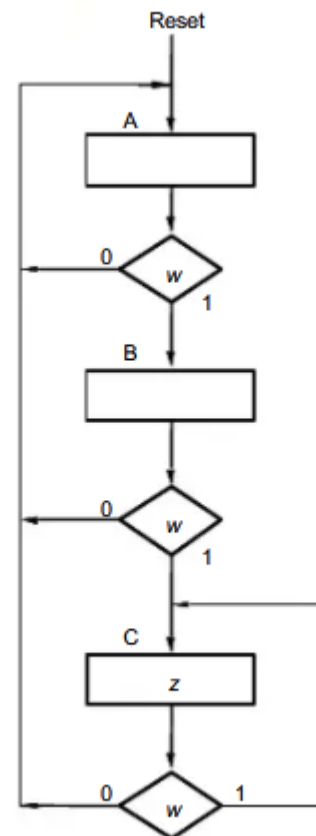
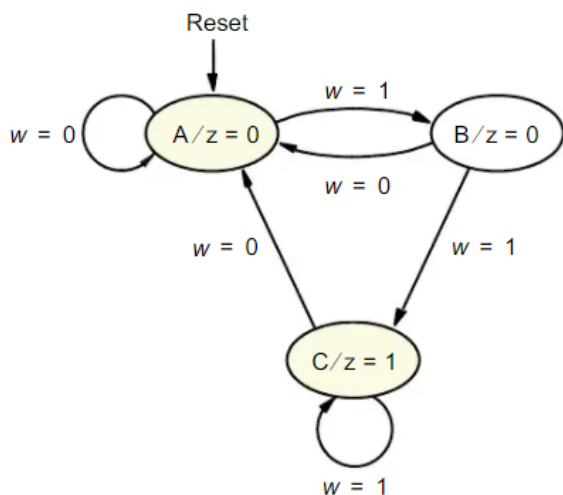
❖ ASM(Algorithm State Machine) Chart

- 순차적인 연산 서술에 적합한 특수한 형태의 Flow Chart로 기존 상태천이도를 제어순서와 데이터패스 연산을 정확하게 표현할 수 있도록 개선
- State Box, Decision Box, Condition Box의 3가지 기본 요소로 구성되며, Box 안의 출력신호, 레지스터 연산은 다음 상태로 천이 시 처리
 - ✓ State Box : 상태이름, 할당 이진코드, 해당 상태의 출력신호, 레지스터 연산(Moore)
 - ✓ Decision Box : 외부입력 및 상태입력에 의한 분기
 - ✓ Condition Box : 반드시 Decision Box에서 분기되어야 하며, Mealy 유형의 출력신호, 레지스터 연산 결과 기술



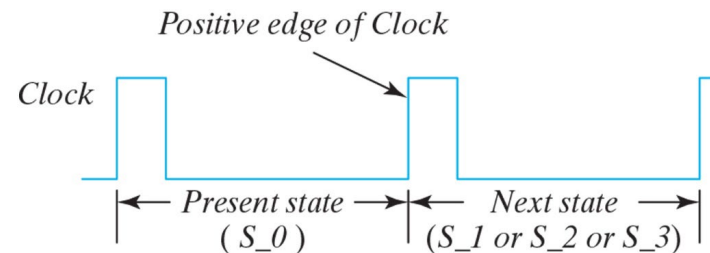
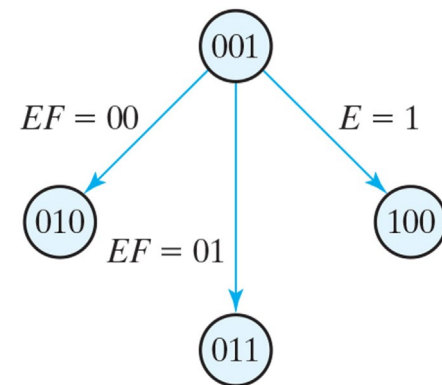
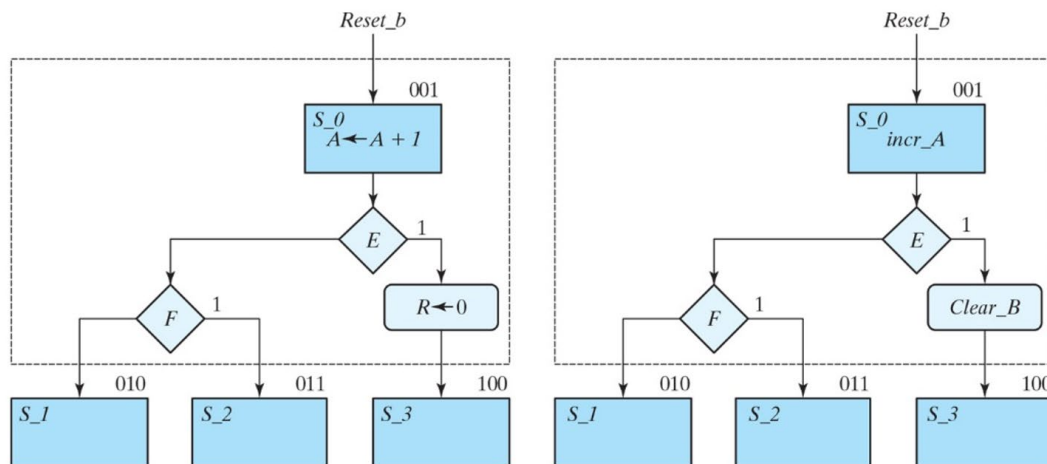


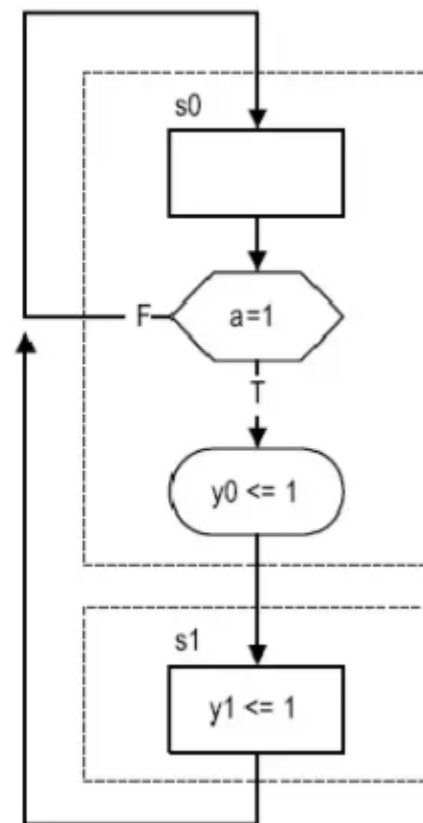
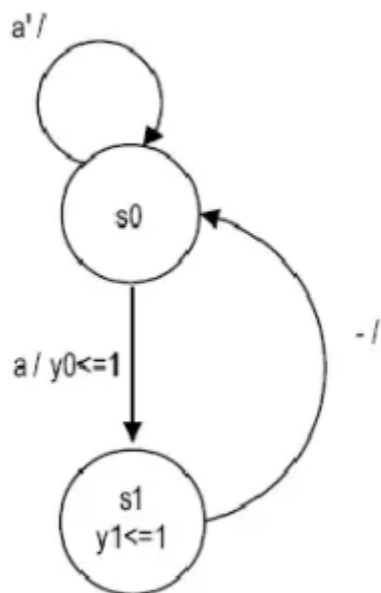
Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	A	B	0
B	A	C	0
C	A	C	1

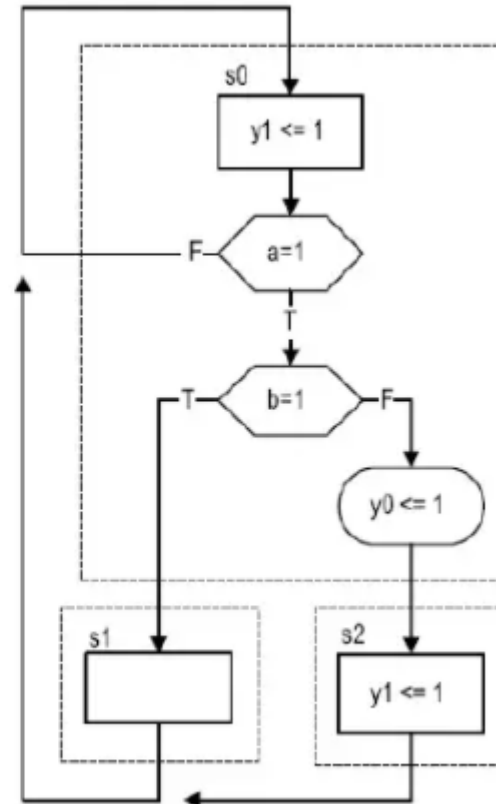
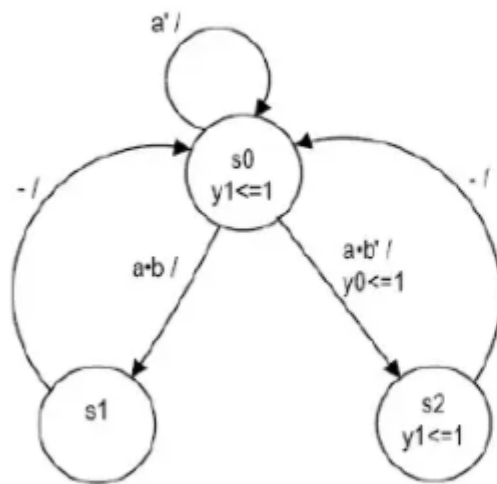


❖ ASM Block

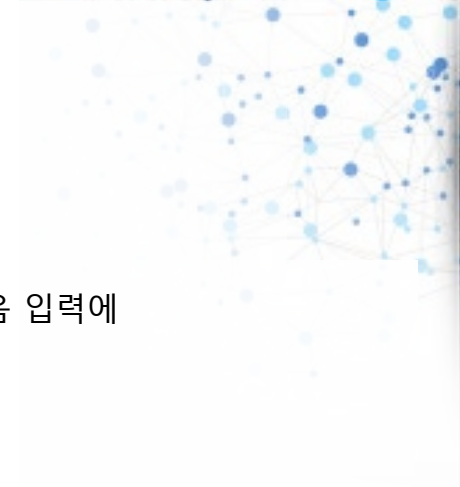
- State Box 1개와 연결된 Decision Box, Condition Box로 이루어진 모듈을 칭함
- Decision Box에 의해 분기되는 경로의 수는 제한이 없음





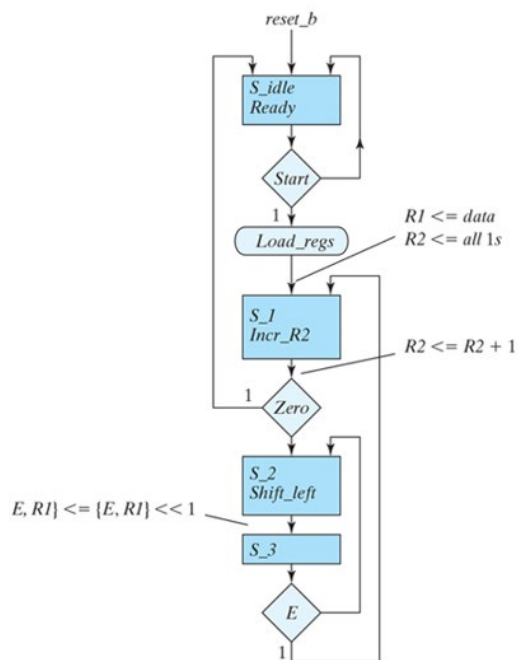


[예제 8.10] 입력 x_{in} 을 모니터링 하여 1이 연속 3번 입력되면 출력 y_{out} 이 1이 된 다음 입력에 0이 입력될 때 까지 유지하는 동기식 상태 머신에 대한 ASM Chart 작성



❖ ASMD(Algorithm State Machine and Datapath) Chart

- 레지스터 연산을 상태가 아닌 상태 천이 관점에서 기술한 방식
- 레지스터 연산과 관련된 처리를 레지스터 연산 실행신호로 표기
- ASM Chart에 비해 레지스터 연산 간의 타이밍이나 레지스터 연산 명령 신호 구분이 용이함





[ASMD Chart 설계 절차]

- 1) Controller의 State Box, Decision Box 및 상태 천이를 발생시키는 입력신호만 나타내는 ASM Chart 작성
- 2) Datapath Unit의 레지스터 연산(상태 천이에 의한 레지스터 연산)을 나타내기 위해 ASM Chart에 주석을 표기하여 ASMD로 변환
- 3) Controller에서 생성된 신호로 Datapath에 표시된 연산이 처리되도록 ASMD 차트 수정

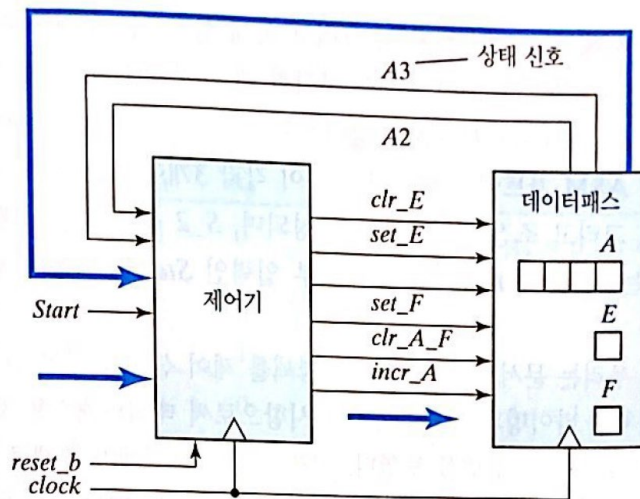
❖ ASMD 설계 예제

[Datapath Unit]

- JK-F/F 2개(E, F)
- 4-bit 2진 카운터 A[3:0], A3가 MSB
- Start 신호로 카운터 A, F/F F가 클리어 되면서 시작
- 매 클럭 펄스마다 1씩 증가
- Reset_bar=0 일때, 초기화

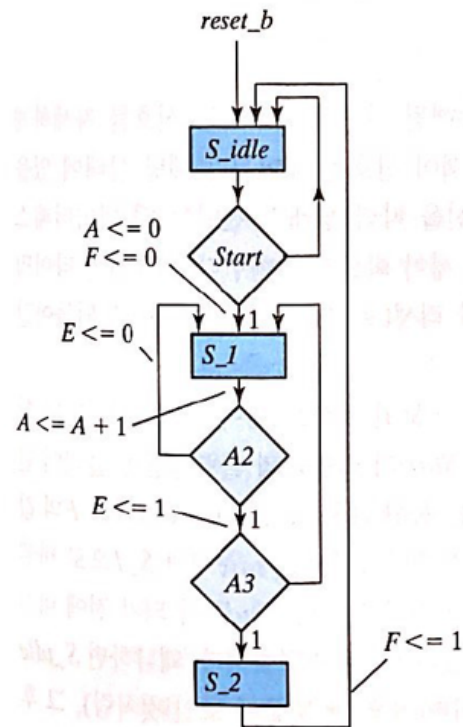
[Control Unit]

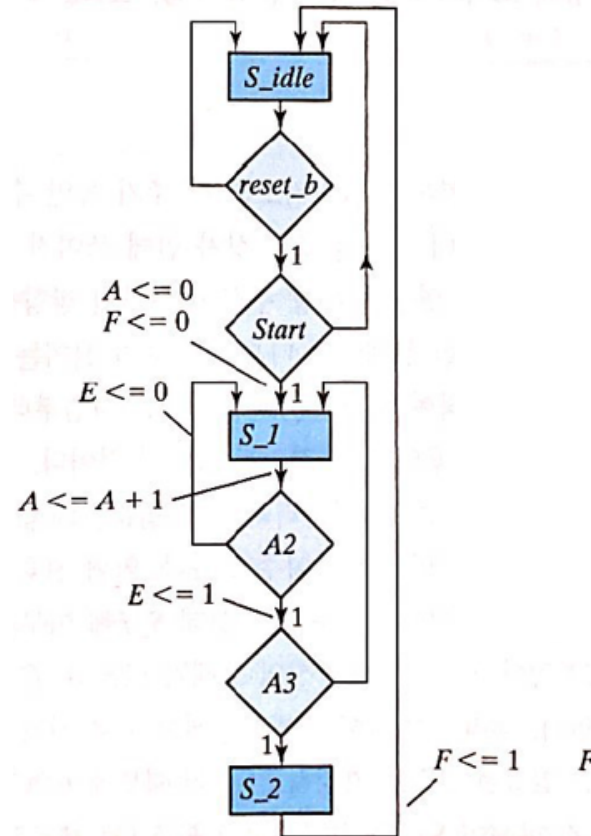
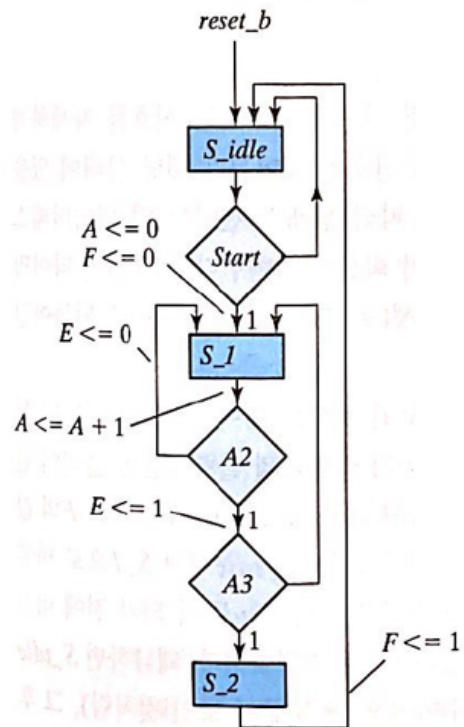
- (A2=0) E=0, 카운트 계속
- (A2=1) E=1 처리 후, A3=0이면 카운트 계속, A3=1이면 다음 클럭에서 F=1, 카운트 중지
- Start=0이면 초기상태에서 정지, Start=1일 경우 시스템 동작



[Control Unit]

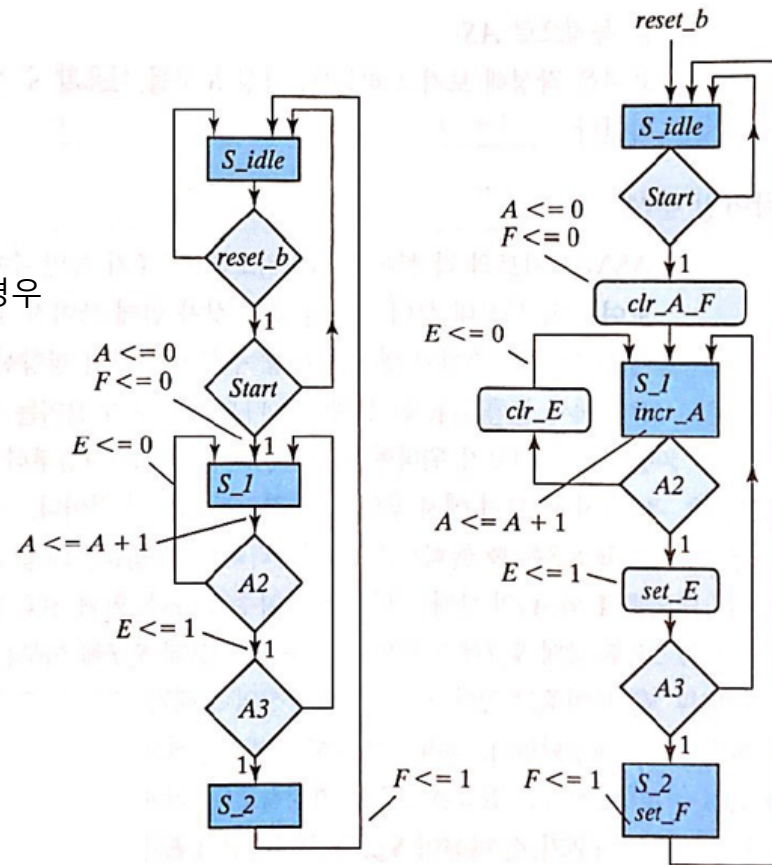
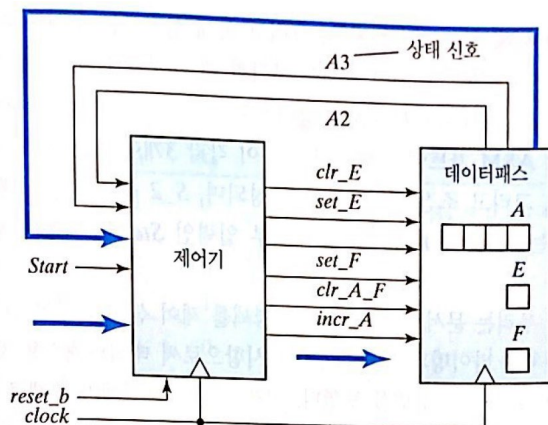
- ($A2=0$) $E=0$, 카운트 계속
- ($A2=1$) $E=1$ 처리 후, $A3=0$ 이면 카운트 계속, $A3=1$ 이면 다음 클럭에서 $F=1$, 카운트 중지
- $Start=0$ 이면 초기상태에서 정지, $Start=1$ 일 경우 시스템 동작

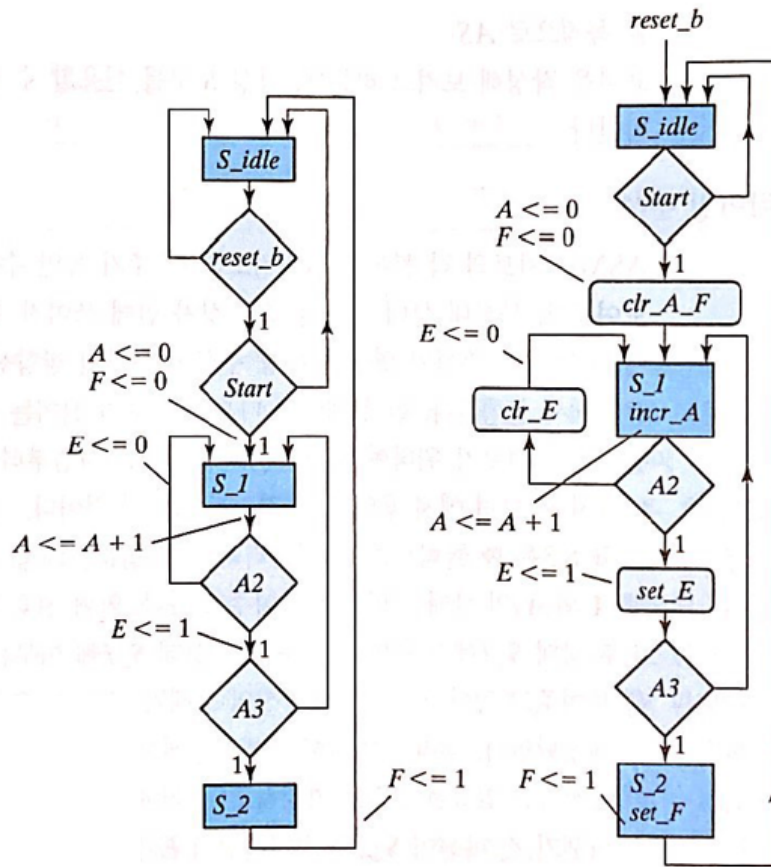




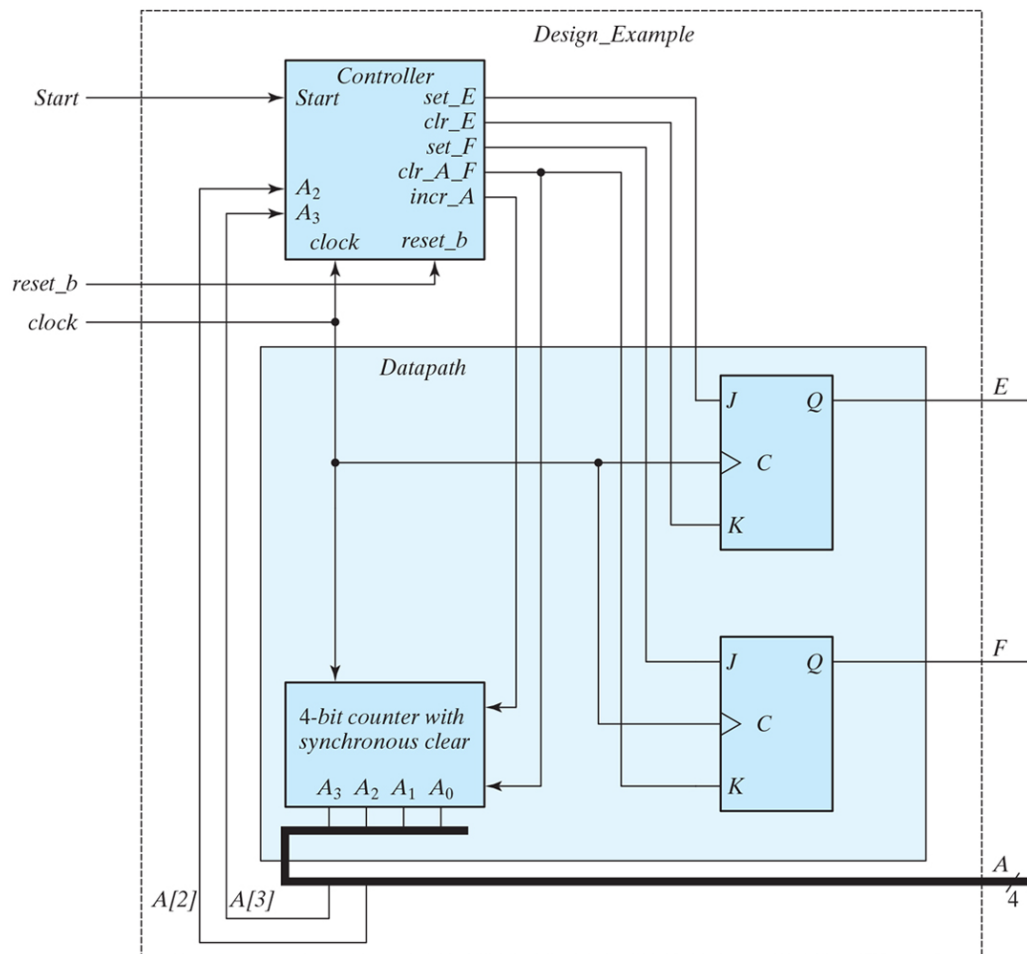
[Control Unit]

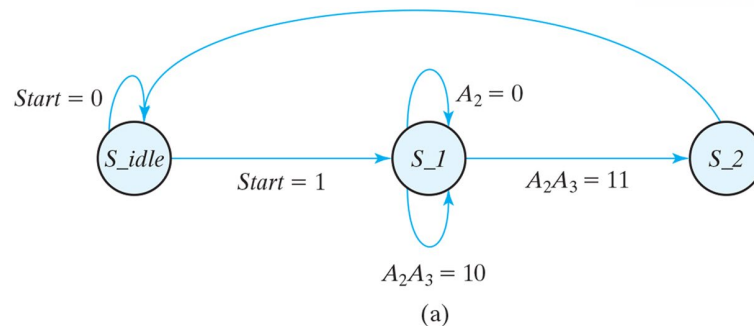
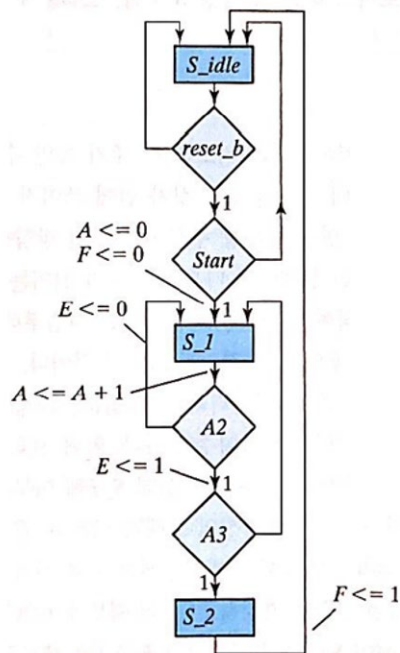
- ($A2=0$) $E=0$, 카운트 계속
- ($A2=1$) $E=1$ 처리 후, $A3=0$ 이면 카운트 계속, $A3=1$ 이면 다음 클럭에서 $F=1$, 카운트 중지
- $Start=0$ 이면 초기상태에서 정지, $Start=1$ 일 경우 시스템 동작





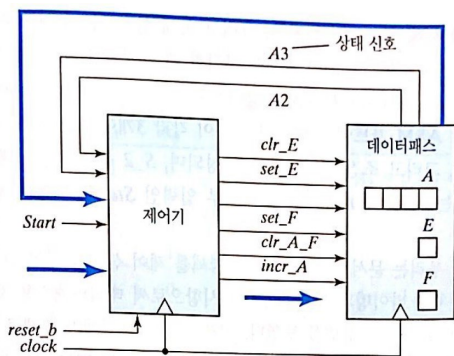
Counter				Flip-Flops		Conditions	State
A ₃	A ₂	A ₁	A ₀	E	F		
0	0	0	0	1	0	A ₂ = 0, A ₃ = 0	S ₁
0	0	0	1	0	0		
0	0	1	0	0	0		
0	0	1	1	0	0		
0	1	0	0	0	0	A ₂ = 1, A ₃ = 0	
0	1	0	1	1	0		
0	1	1	0	1	0		
0	1	1	1	1	0		
1	0	0	0	1	0	A ₂ = 0, A ₃ = 1	
1	0	0	1	0	0		
1	0	1	0	0	0		
1	0	1	1	0	0		
1	1	0	0	0	0	A ₂ = 1, A ₃ = 1	S ₂
1	1	0	1	1	0		
1	1	0	1	1	1		
1	1	0	1	1	1		S _{idle}



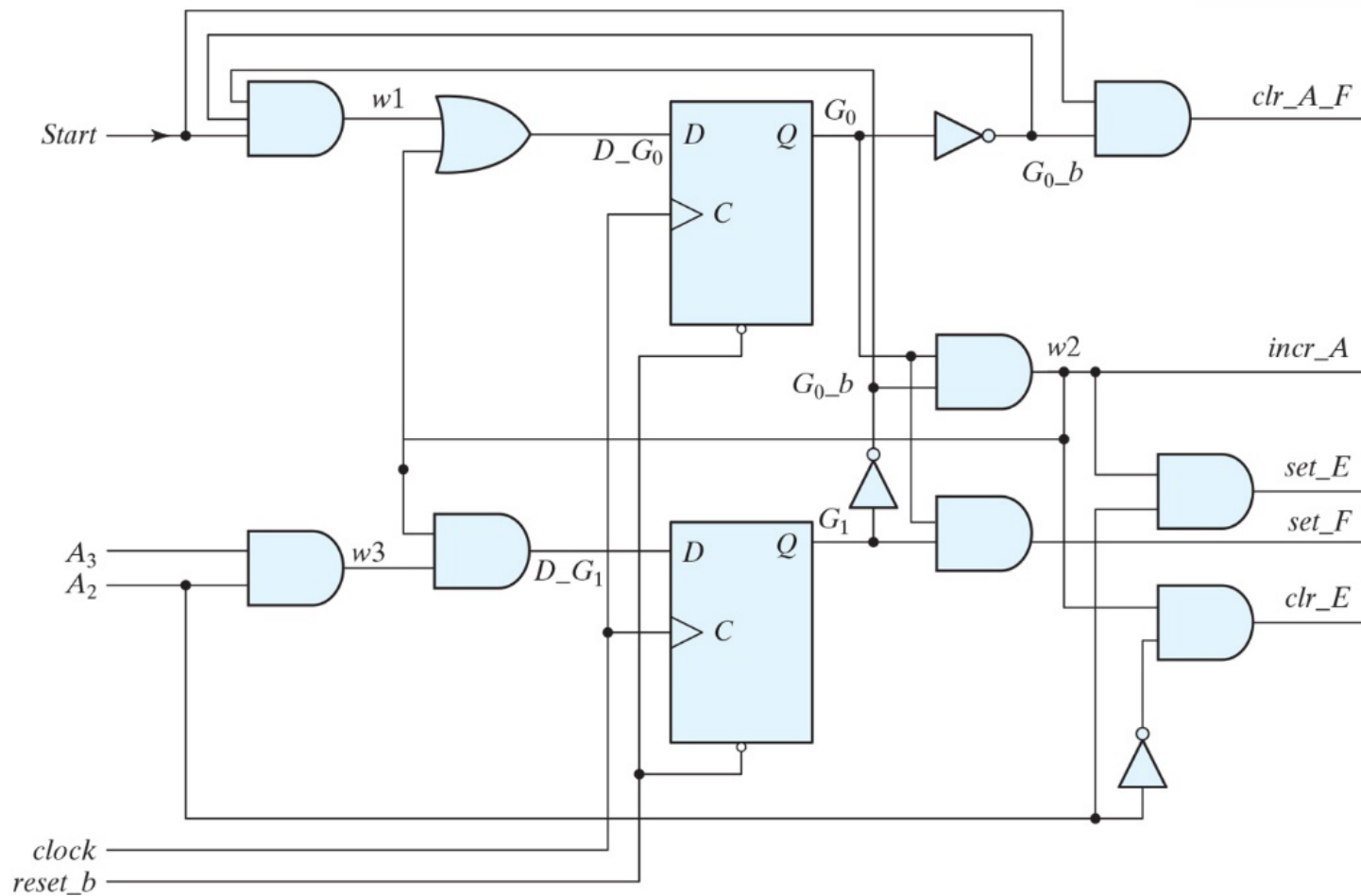


$S_idle \rightarrow S_1, clr_A_F:$ $A \leftarrow 0, F \leftarrow 0$
 $S_1 \rightarrow S_1, incr_A:$ $A \leftarrow A + 1$
 if ($A_2 = 1$) then set_E: $E \leftarrow 1$
 if ($A_2 = 0$) then clr_E: $E \leftarrow 0$
 $S_1 \rightarrow S_2, incr_A:$ $F \leftarrow 1$
 $S_2 \rightarrow S_idle:$ $A \leftarrow A + 1$

(b)



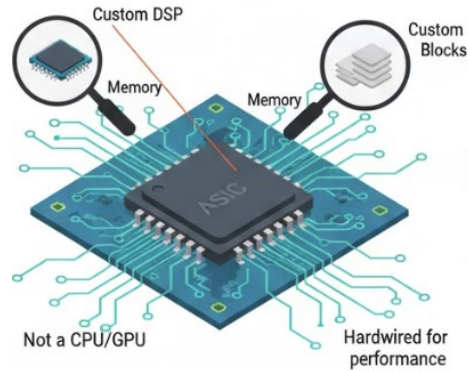
	Present State		Inputs			Next State		Outputs				
	G ₁	G ₀	Start	A ₂	A ₃	G ₁	G ₀	set_E	clr_E	set_F	clr_A_F	incr_A
Present-State Symbol												
S_idle	0	0	0	X	X	0	0	0	0	0	0	0
S_idle	0	0	1	X	X	0	1	0	0	0	1	0
S_1	0	1	X	0	X	0	1	0	1	0	0	1
S_1	0	1	X	1	0	0	1	1	0	0	0	1
S_1	0	1	X	1	1	1	1	1	0	0	0	1
S_2	1	1	X	X	X	0	0	0	0	1	0	0



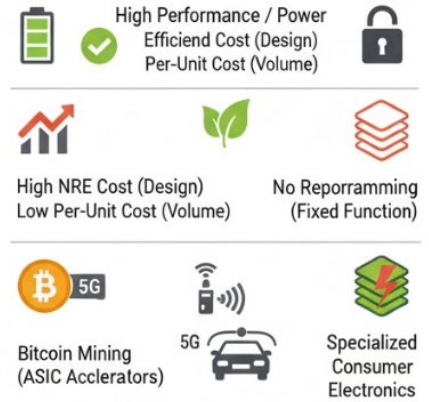
❖ ASIC (Application Specific IC)

- 소량의 특정 목적의 IC
- 고성능 설계 도구의 영향
- 회로 직접도 증가의 결과
- 하나의 칩에 전체 시스템 직접 가능
- 설계 문제 복잡도가 기하 급수적으로 증가
- 설계 기법 및 도구의 성능 향상
 - HDL (Hardware Description Language)
 - Logic Synthesis System
- 설계가 Layout, Gate 수준에서의 RTL 수준 이상으로 높아짐
 - Logic Synthesis
 - Layout Synthesis
 - Placement & Routing

What is it?



Key Features & Uses



Level	Graphical	Representation
System Partitioning, Pipelining Behavioral Scheduling, Allocation		HDL, Graphical specifications
RTL State Machines, Logic and RTL Multi-level, 2-level Optimization		
Logic Technology mapping/ translation		Logic equations, HDL, Gates
Implementation Place and Route		Layout CIF/GDSII

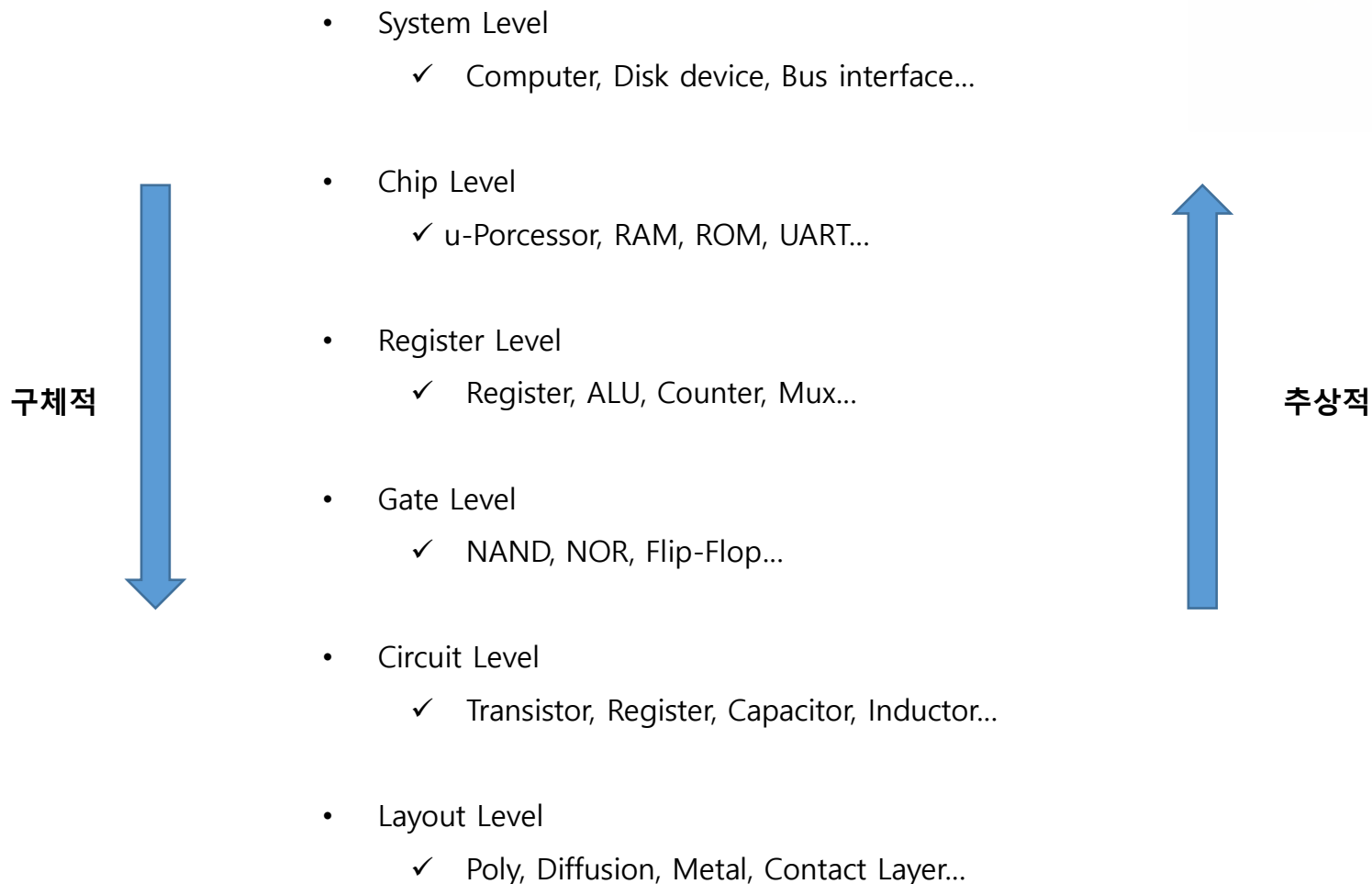
```

.....
ARCHITECTURE rtl OF counter IS
  SIGNAL incount : UNSIGNED(7 DOWNTO 0);
BEGIN
  PROCESS(clk,rst)
  BEGIN
    IF( rst = '1') THEN
      incount <= "00000000";
    ELSIF (clk'EVENT AND clk='1') THEN
      incount <= incount + '1';
    END IF;
  END PROCESS;
END rtl;
.....

```



❖ Top-Down 방식 설계



❖ 순차 2진 곱셈기 설계

- 부호가 없는 2진수 2개의 곱 처리
 - 1) 사양 결정 및 알고리즘 결정
 - 2) 구현을 위한 시스템 구성 결정 (Controller, Data Path, Control & Data line)
 - 3) ASMD Chart
 - 4) 상태 천이도
 - 5) 상태 천이 별 제어 상세 내용 정의
 - 6) 제어회로 상태표 작성
 - 7) 제어회로 설계
 - 8) Data Path 설계

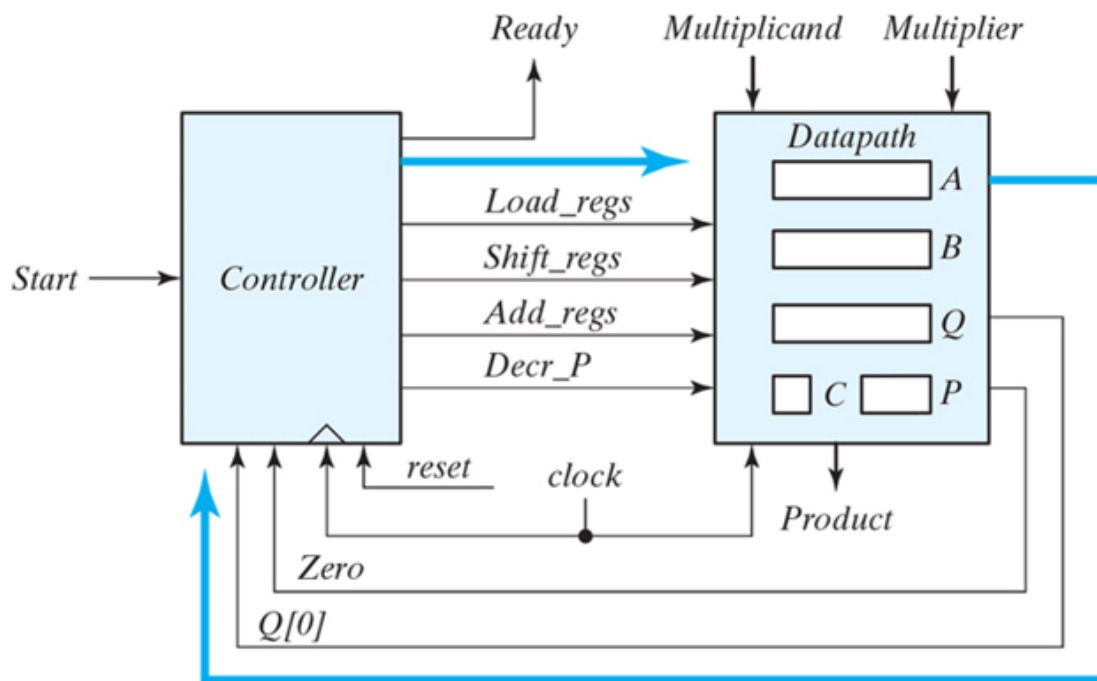


❖ 순차 2진 곱셈기 설계

$$\begin{array}{r}
 \begin{array}{r}
 \times \\
 \hline
 \end{array}
 \begin{array}{r}
 1\ 0\ 1\ 1\ 1 \\
 1\ 0\ 0\ 1\ 1 \\
 \hline
 1\ 0\ 1\ 1\ 1 \\
 1\ 0\ 1\ 1\ 1 \\
 0\ 0\ 0\ 0\ 0 \\
 0\ 0\ 0\ 0\ 0 \\
 +\ 1\ 0\ 1\ 1\ 1 \\
 \hline
 1\ 1\ 0\ 1\ 1\ 0\ 1\ 0\ 1
 \end{array}
 \begin{array}{l}
 (\text{피승수: } 23) \\
 (\text{승수: } 19) \\
 (\times 1) \\
 (\times 1, 1\text{칸 시프트}) \\
 (\times 0, 2\text{칸 시프트}) \\
 (\times 0, 3\text{칸 시프트}) \\
 (\times 1, 4\text{칸 시프트}) \\
 (\text{결과값: } 437)
 \end{array}
 \end{array}$$



❖ 순차 2진 곱셈기 설계



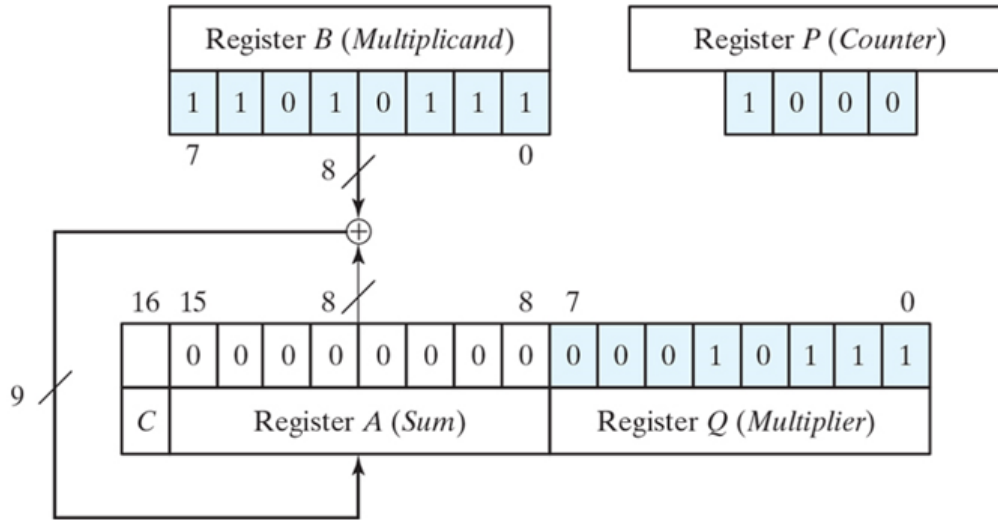
피승수 → B

승수 → Q

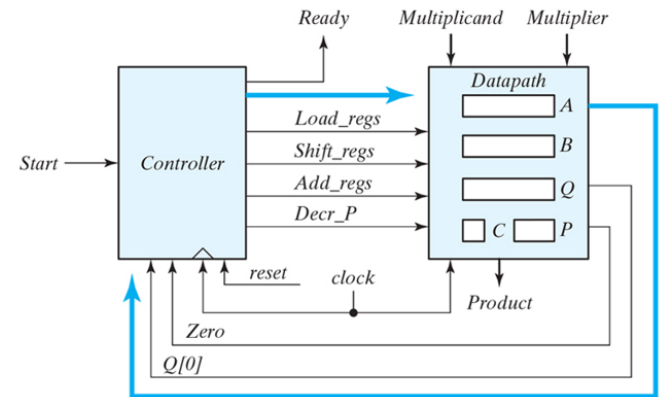
승수의 비트 수 → P (Down Counter)

부분곱 → A

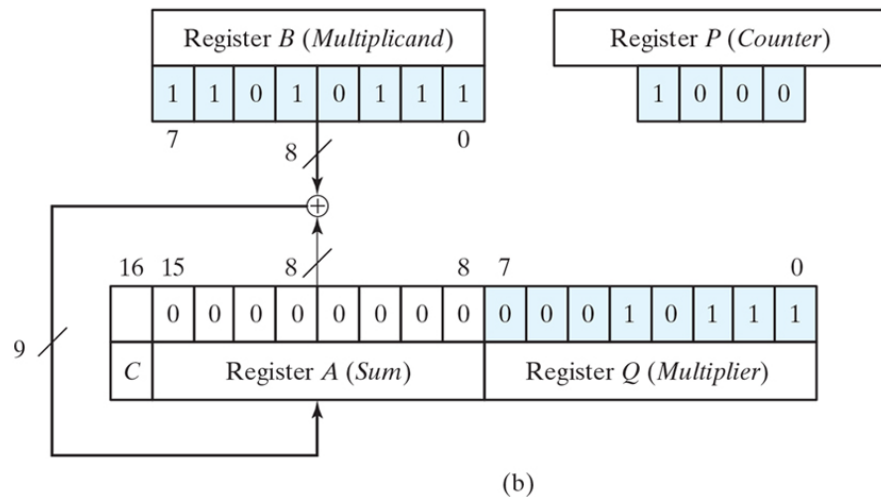
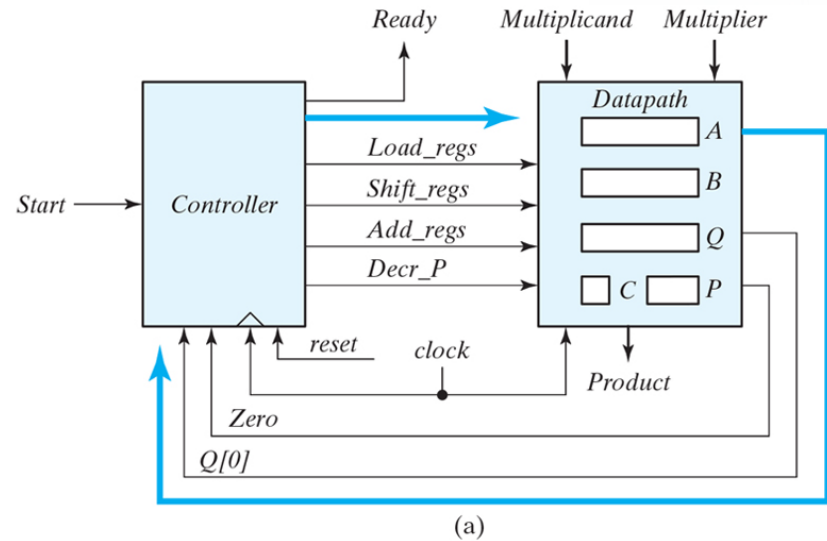
부분곱의 합에서 발생하는 올림 → C

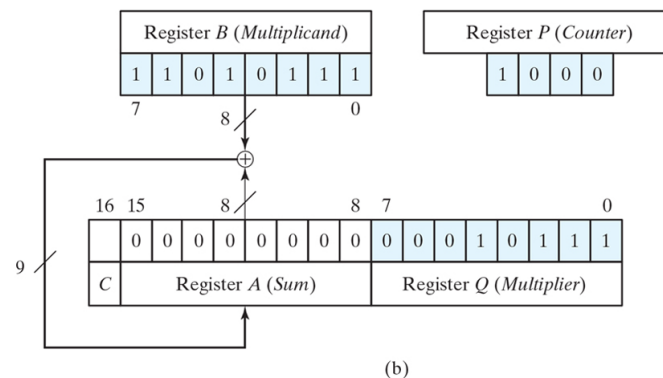
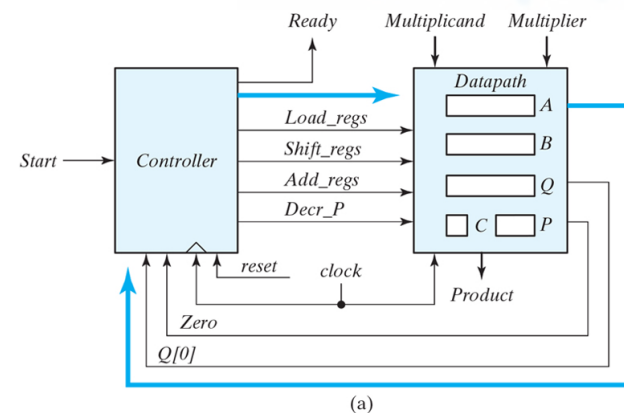
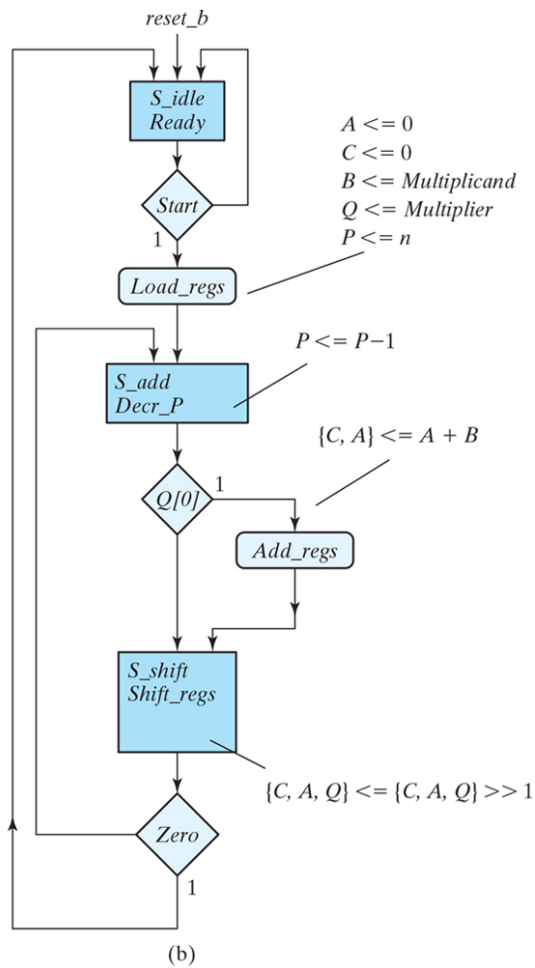
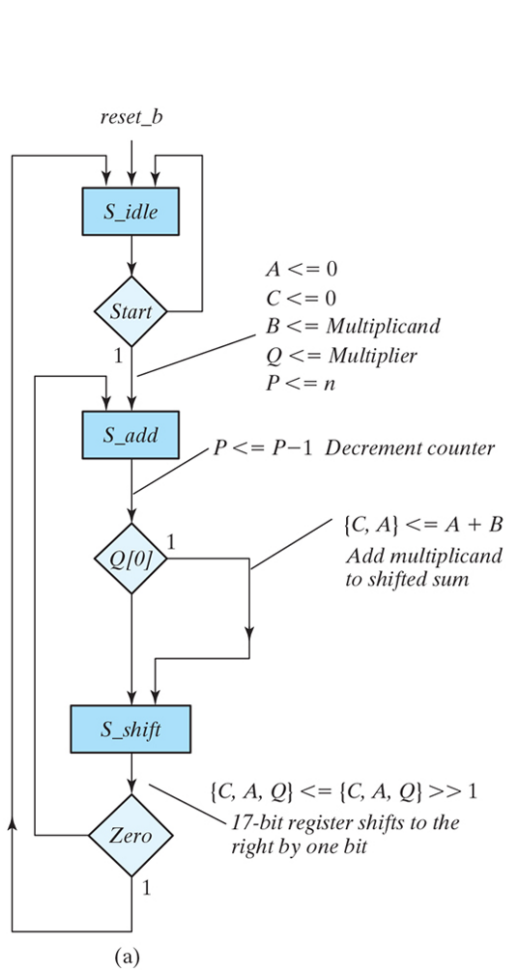


		1 1 0 1 0 1 1 1	(피승수: 215)
×		0 0 0 1 0 1 1 1	(승수: 23)
		1 1 0 1 0 1 1 1	(×1)
	1 1 0 1 0 1 1 1		(×1, 1칸 시프트)
	1 1 0 1 0 1 1 1		(×1, 2칸 시프트)
	0 0 0 0 0 0 0 0		(×0, 3칸 시프트)
+	1 1 0 1 0 1 1 1		(×1, 4칸 시프트)
	1 0 0 1 1 0 1 0 1 0 0 0 1		(결과값: 4945)



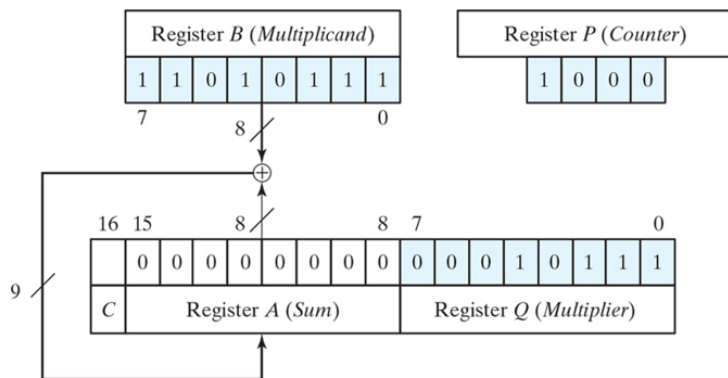
State	Binary
S_idle	00
S_add	01
S_shift	10



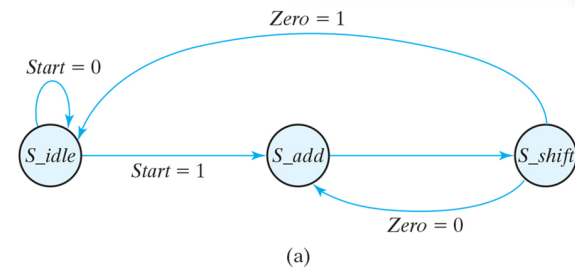
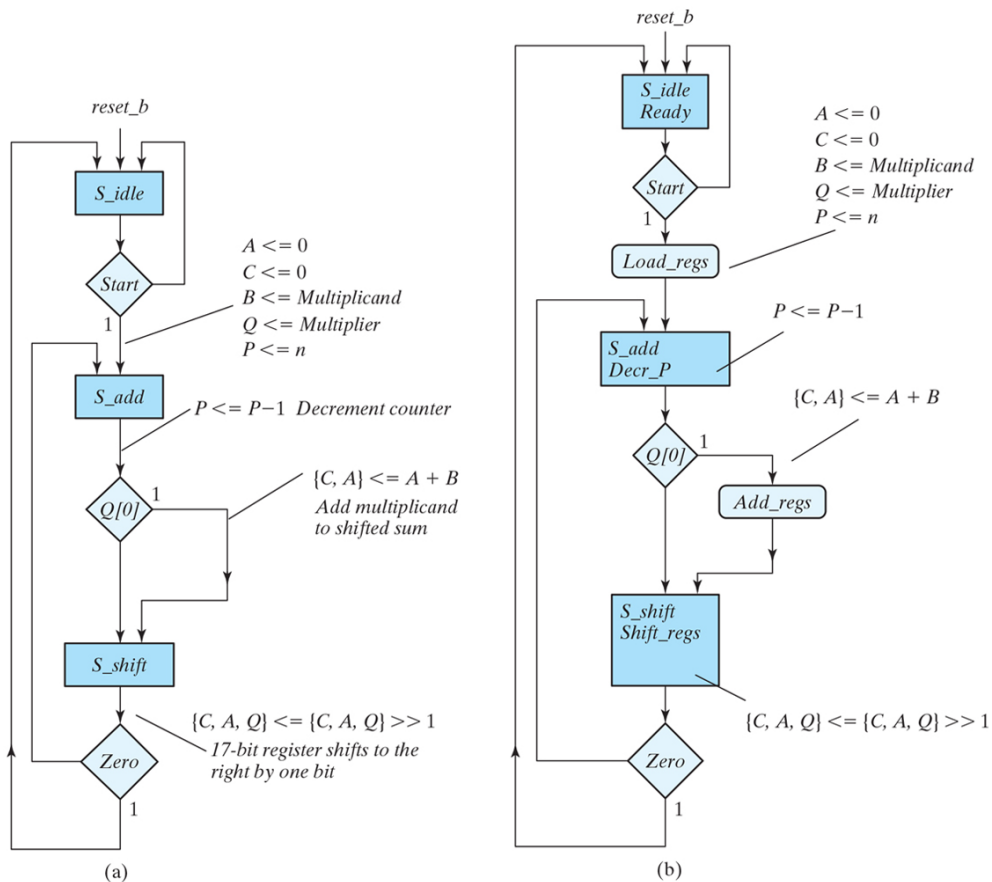


Multiplicand $B = 10111_2 = 17_H = 23_{10}$ Multiplier $Q = 10011_2 = 13_H = 19_{10}$

	C	A	Q	P
Multiplicand in Q	0	00000	10011	101
$Q_0 = 1$; add B		<u>10111</u>		
First partial product	0	10111		100
Shift right CAQ	0	01011	11001	
$Q_0 = 1$; add B		<u>10111</u>		
Second partial product	1	00010		011
Shift right CAQ	0	10001	01100	
$Q_0 = 0$; shift right CAQ	0	01000	10110	010
$Q_0 = 0$; shift right CAQ	0	00100	01011	001
$Q_0 = 1$; add B		<u>10111</u>		
Fifth partial product	0	11011		
Shift right CAQ	0	01101	10101	000
Final product in $AQ = 0110110101_2 = 1b5_H$				



				1	0	1	1	1	(피승수: 23)
×				1	0	0	1	1	(승수: 19)
				1	0	1	1	1	(×1)
			1	0	1	1	1		(×1, 1칸 시프트)
		0	0	0	0	0			(×0, 2칸 시프트)
		0	0	0	0	0			(×0, 3칸 시프트)
+	1	0	1	1	1				(×1, 4칸 시프트)
	1	1	0	1	1	0	1	0	1 (결과값: 437)

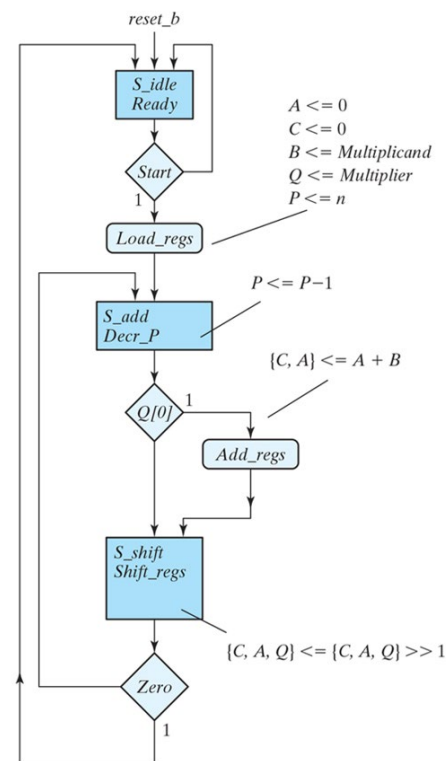
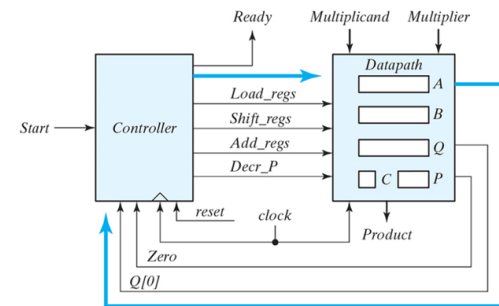
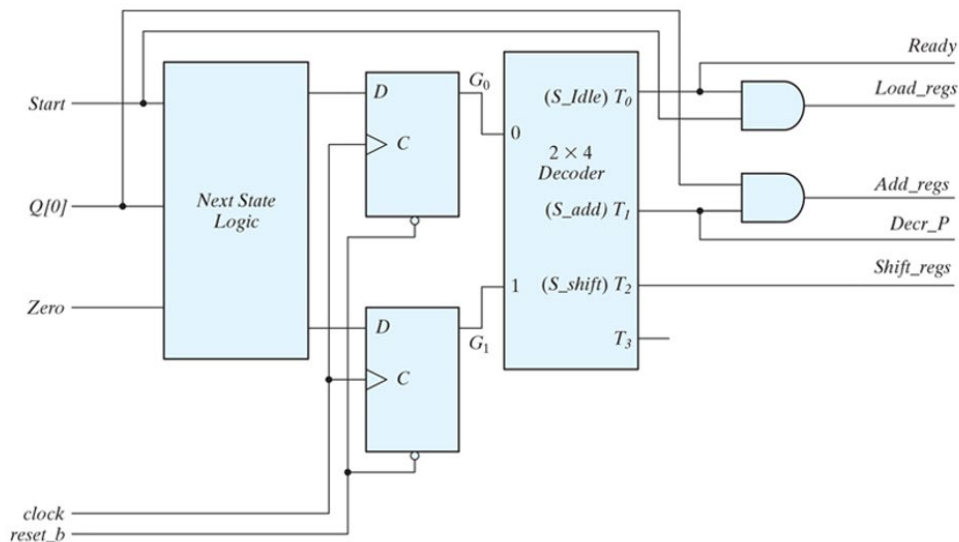


State Transition		Register Operations
From	To	
S_idle	S_idle	Initial state
S_idle	S_add	$A \leq 0$, $C \leq 0$, $P \leq dp_width$ $A \leq \text{Multiplicand}$, $Q \leq \text{Multiplier}$
S_add	S_shift	$P \leq P - 1$ if ($Q[0]$) then ($A \leq A + B$, $C \leq C_{out}$)
S_shift	S_shift	shift right {CAQ}, $C \leq 0$

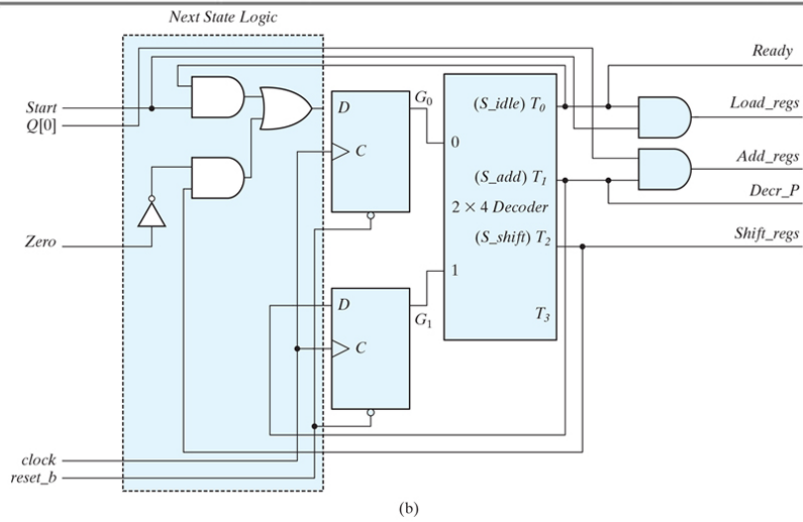
(b)

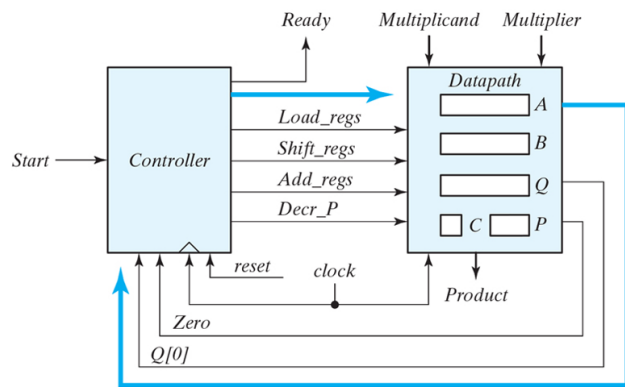
State	Binary	Gray Code	One-Hot
S_idle	00	00	001
S_add	01	01	010
S_shift	10	11	100

Present-State Symbol	Present State		Inputs			Next State		Outputs				
	G_1	G_0	Start	$Q[0]$	Zero	G_1	G_0	Ready	Load_regs	Decr_P	Add_regs	Shift_regs
S_idle	0	0	0	X	X	0	0	1	0	0	0	0
S_idle	0	0	1	X	X	0	1	1	1	0	0	0
S_add	0	1	X	0	X	1	0	0	0	1	0	0
S_add	0	1	X	1	X	1	0	0	0	1	1	0
S_shift	1	0	X	X	0	0	1	0	0	0	0	1
S_shift	1	0	X	X	1	0	0	0	0	0	0	1

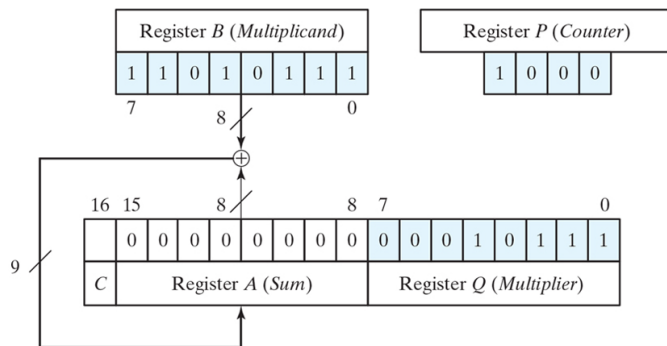


	Present State		Inputs			Next State		Outputs				
Present-State Symbol	G_1	G_0	Start	$Q[0]$	Zero	G_1	G_0	Ready	Load_regs	Decr_P	Add_regs	Shift_regs
S_idle	0	0	0	X	X	0	0	1	0	0	0	0
S_idle	0	0	1	X	X	0	1	1	1	0	0	0
S_add	0	1	X	0	X	1	0	0	0	1	0	0
S_add	0	1	X	1	X	1	0	0	0	1	1	0
S_shift	1	0	X	X	0	0	1	0	0	0	0	1
S_shift	1	0	X	X	1	0	0	0	0	0	0	1

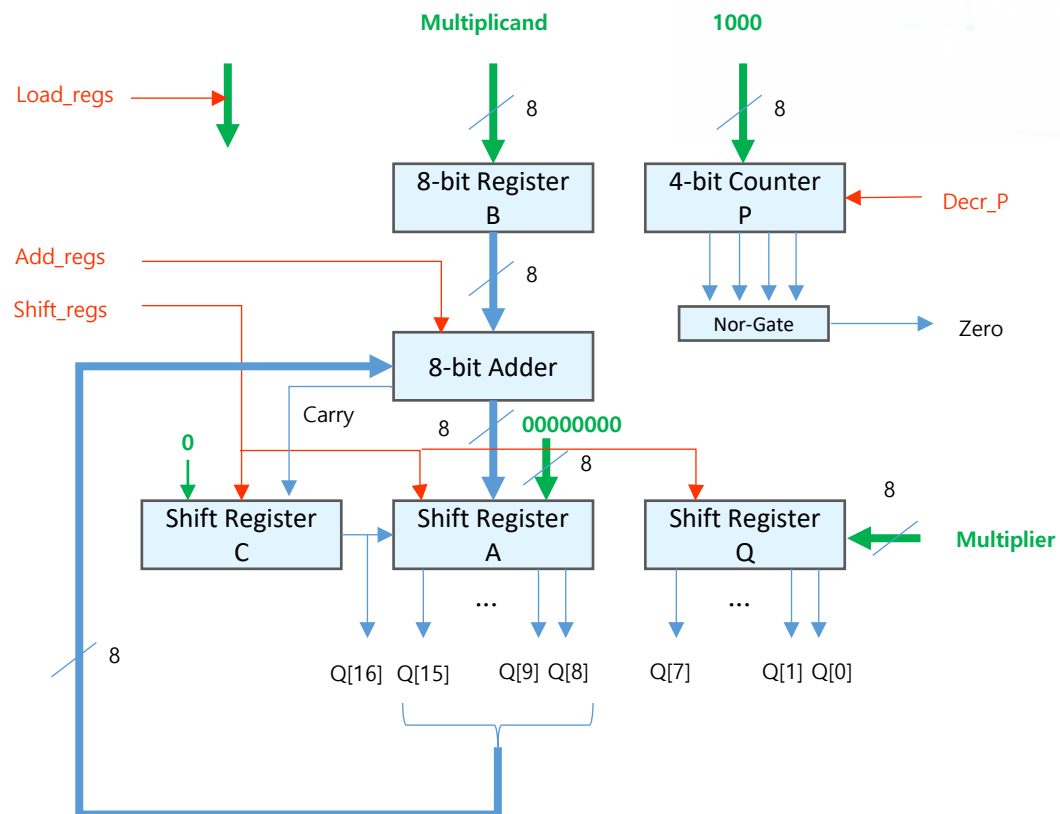


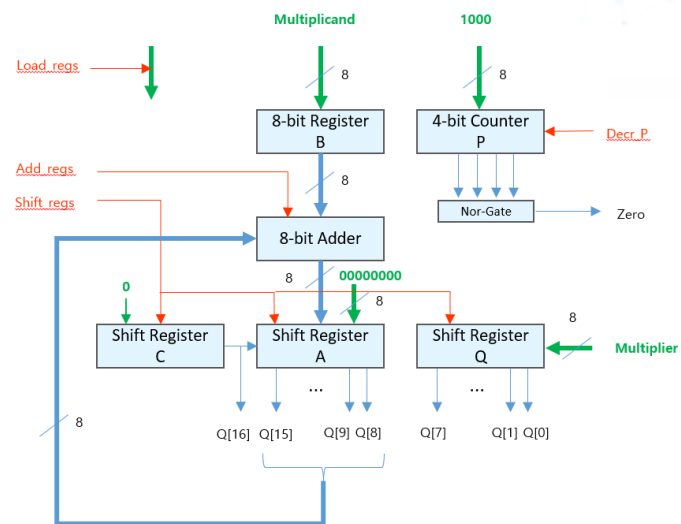
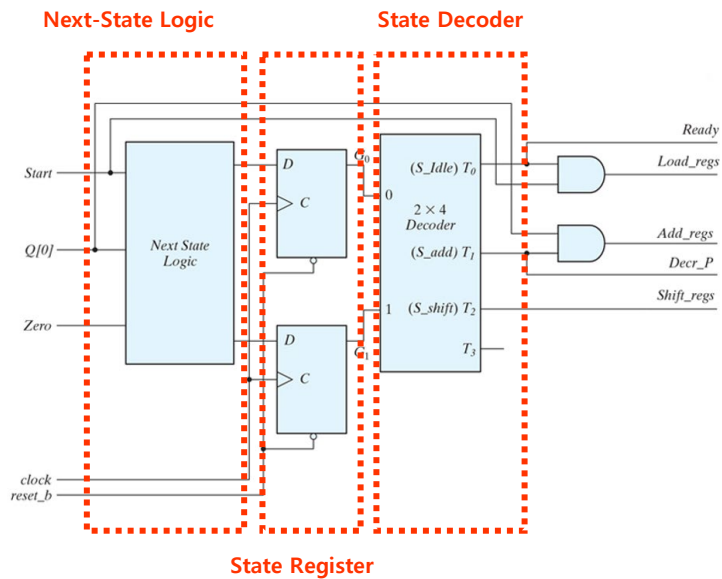
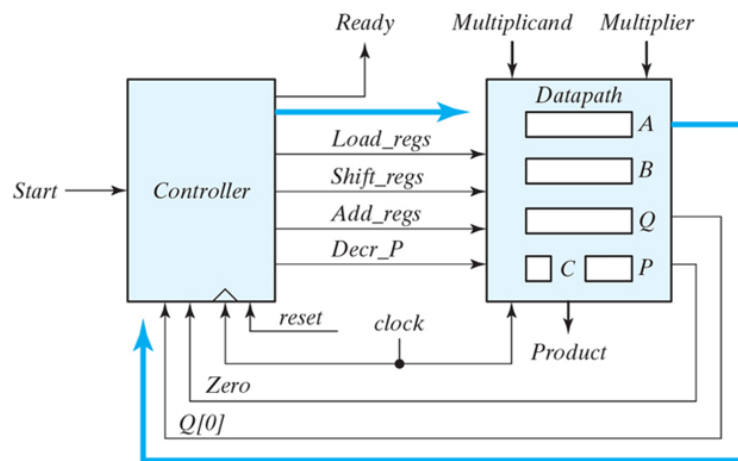


(a)



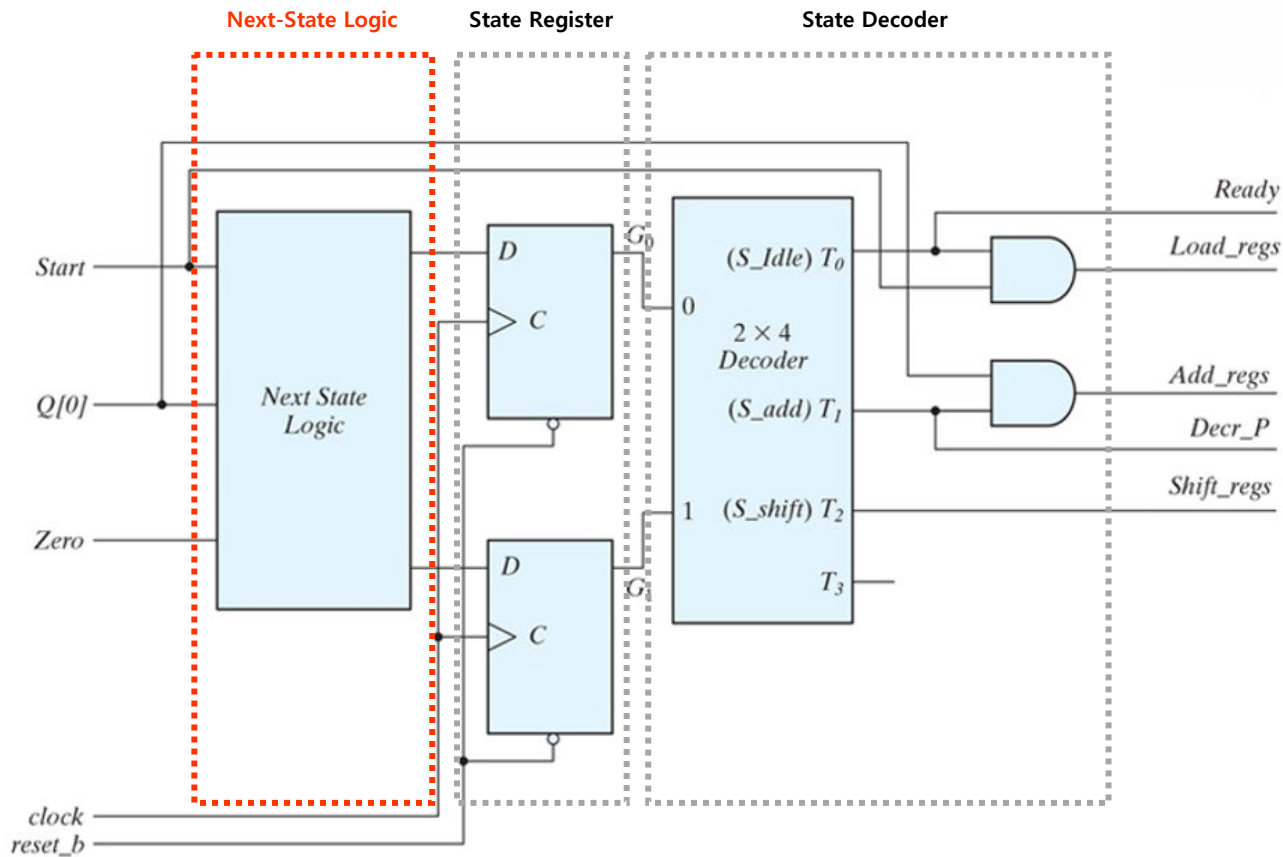
(b)

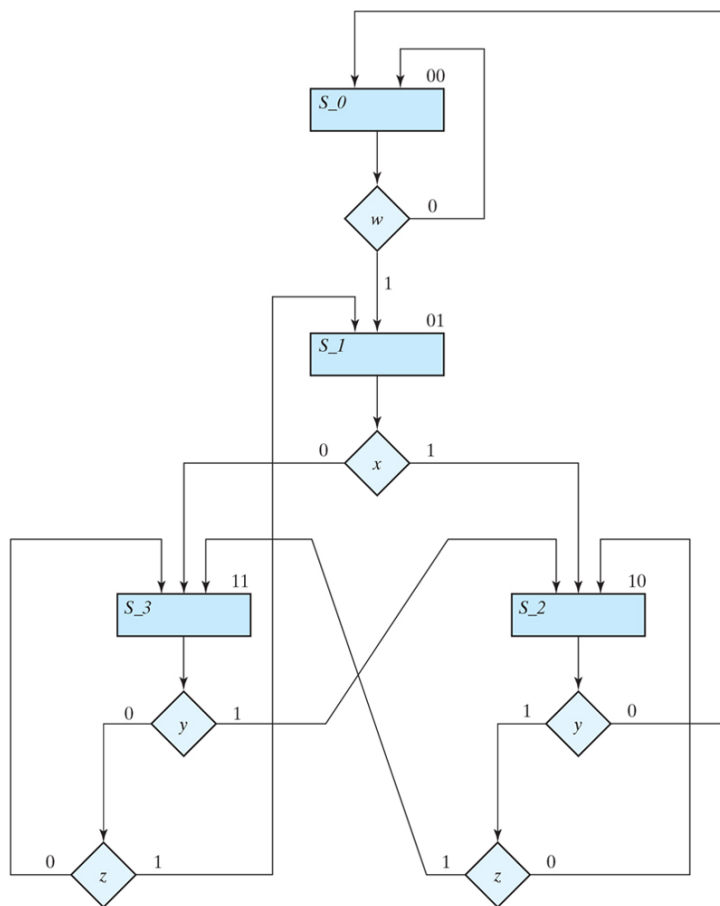




8 레지스터 전송 레벨 설계-3

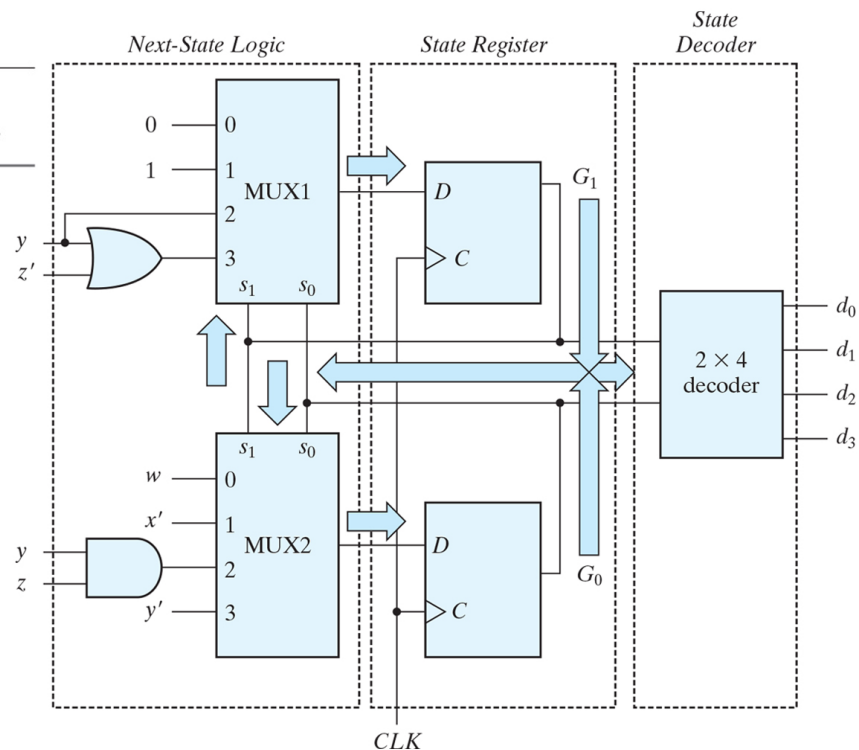
❖ Mux를 사용한 상태전이 로직



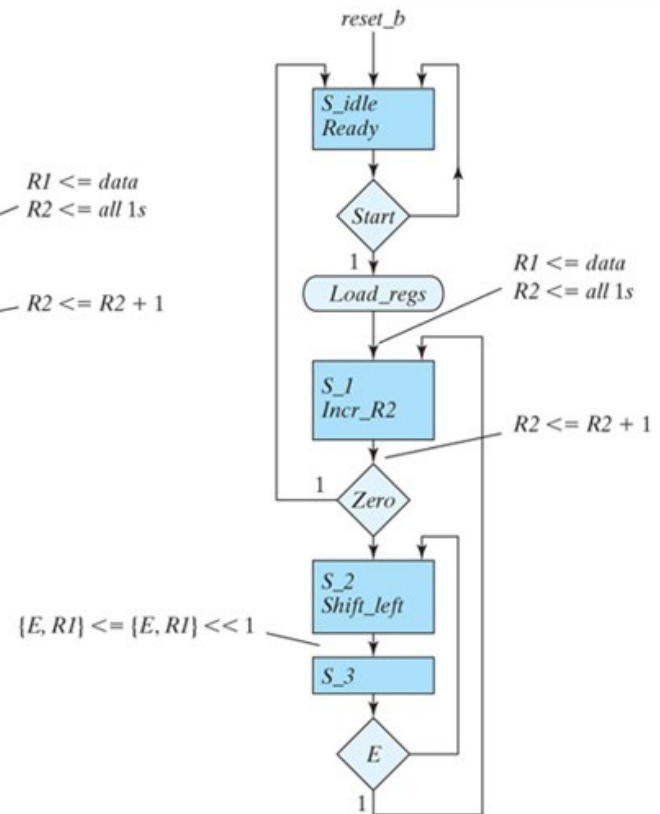
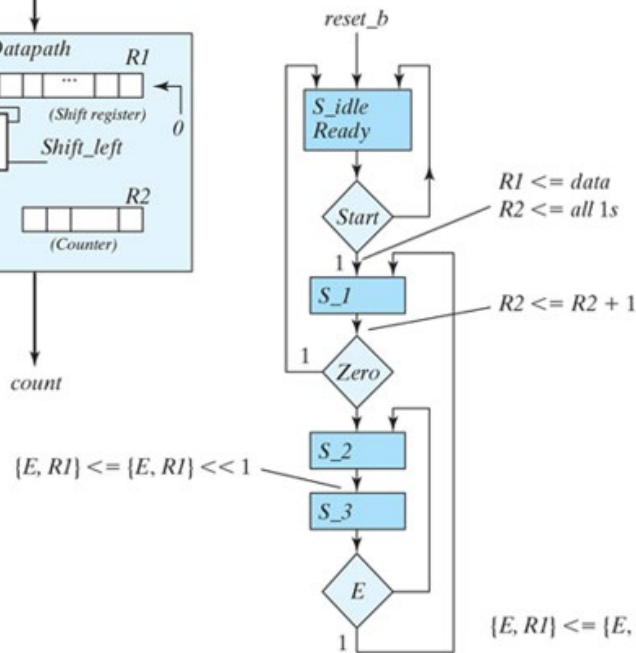
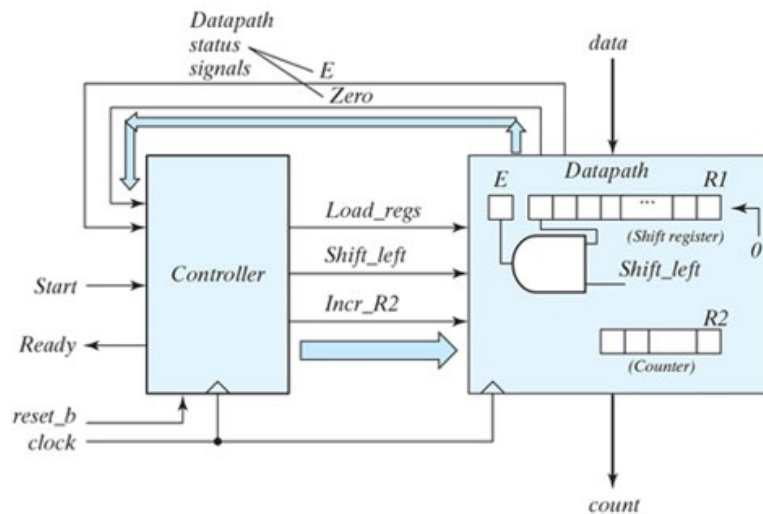


Present State		Next State		Input Condition
G ₁	G ₀	G ₁	G ₀	
0	0	0	0	w'
0	0	0	1	w
0	1	1	0	x
0	1	1	1	x'
1	0	0	0	y'
1	0	1	0	yz'
1	0	1	1	yz
1	1	0	1	y'z
1	1	1	0	y
1	1	1	1	y'z'

Present State		Next State		Input Condition	Inputs	
G_1	G_0	G_1	G_0		MUX1	MUX2
0	0	0	0	w'		
0	0	0	1	w	0	w
0	1	1	0	x		
0	1	1	1	x'	1	x'
1	0	0	0	y'		
1	0	1	0	yz'		
1	0	1	1	yz	$yz' + yz = y$	yz
1	1	0	1	$y'z$		
1	1	1	0	y		
1	1	1	1	$y'z'$	$y + y'z' = y + z'$	$y'z + y'z' = y'$



❖ Register의 Set bit Count 예제

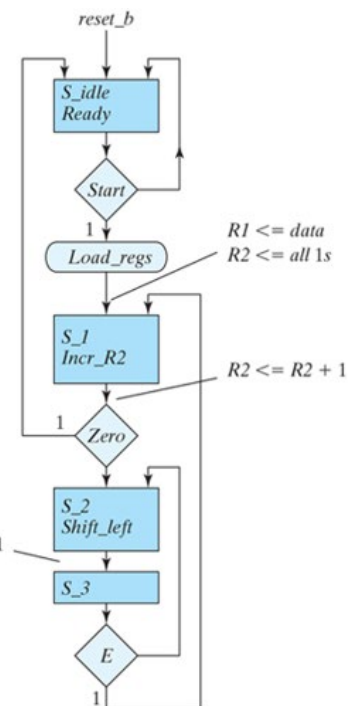
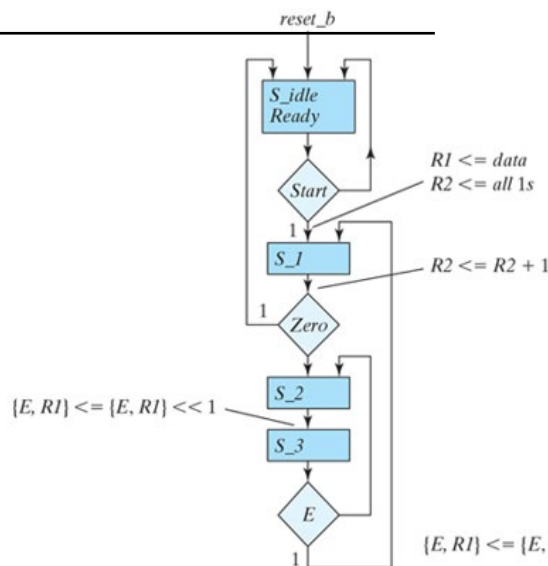
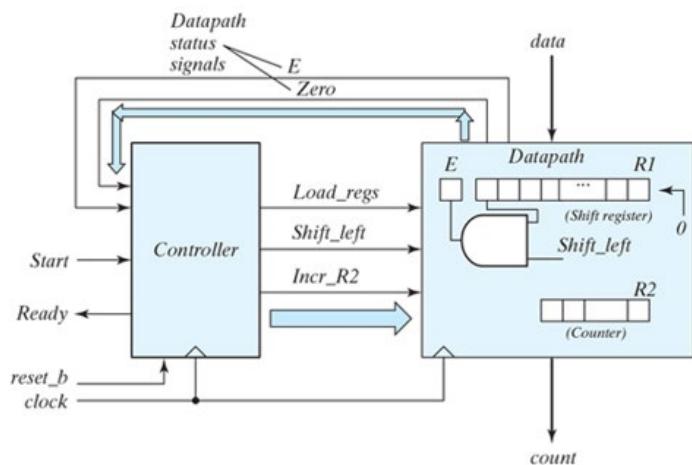


Next-State Logic

Mux 출력

출력

외부 입력			현재상태		다음상태		플립플롭 입력		출력			
Start	Zero	E	G1	G0	G1	G0	D1	D0	Ready	Incr_R2	Shift_left	Load_regs
0	X	X	0	0	0	0	0	0	1			
1	X	X	0	0	0	1	0	1	1			1
X	1	X	0	1	0	0	0	0		1		
X	0	X	0	1	1	0	1	0		1		
X	X	X	1	0	1	1	1	1			1	
X	X	0	1	1	1	0	1	0				
X	X	1	1	1	0	1	0	1				



Next-State Logic

Mux 출력

외부 입력			현재상태		다음상태		플립플롭 입력		출력			
Start	Zero	E	G1	G0	G1	G0	D1	D0	Ready	Incr_R2	Shift_left	Load_regs
0	X	X	0	0	0	0	0	0	1			
1	X	X	0	0	0	1	0	1	1			1
X	1	X	0	1	0	0	0	0		1		
X	0	X	0	1	1	0	1	0		1		
X	X	X	1	0	1	1	1	1			1	
X	X	0	1	1	1	0	1	0				
X	X	1	1	1	0	1	0	1				

외부 입력			현재상태		다음상태		플립플롭 입력	
Start	Zero	E	G1	G0	G1	G0	D1	D0
0	X	X	0	0	0	0	0	0
1	X	X	0	0	0	1	0	1
X	1	X	0	1	0	0	0	0
X	0	X	0	1	1	0	1	0
X	X	X	1	0	1	1	1	1
X	X	0	1	1	1	0	1	0
X	X	1	1	1	0	1	0	1

Present State		Next State		Input Conditions	Multiplexer Inputs	
G ₁	G ₀	G ₁	G ₀		MUX1	MUX2
0	0	0	0	Start'	0	Start
0	0	0	1	Start		
0	1	0	0	Zero	Zero'	0
0	1	1	0	Zero'		
1	0	1	1	None	1	1
1	1	1	0	E'	E'	E
1	1	0	1	E		

State Decoder

Mux 출력

Controller Output

외부 입력			현재상태		다음상태		플립플롭 입력		출력			
Start	Zero	E	G1	G0	G1	G0	D1	D0	Ready	Incr_R2	Shift_left	Load_regs
0	X	X	0	0	0	0	0	0	1			
1	X	X	0	0	0	1	0	1	1			1
X	1	X	0	1	0	0	0	0		1		
X	0	X	0	1	1	0	1	0		1		
X	X	X	1	0	1	1	1	1			1	
X	X	0	1	1	1	0	1	0				
X	X	1	1	1	0	1	0	1				

Present State		Next State		Input Conditions	Multiplexer Inputs	
G ₁	G ₀	G ₁	G ₀		MUX1	MUX2
0	0	0	0	Start'		
0	0	0	1	Start	0	Start
0	1	0	0	Zero		
0	1	1	0	Zero'	Zero'	0
1	0	1	1	None	1	1
1	1	1	0	E'		
1	1	0	1	E	E'	E

